

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250286842

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Alsina; Thomas M. et al.

CONVERSION OF TEXT RELATING TO MEDIA CONTENT AND MEDIA EXTENSION APPS

Abstract

A first messaging app, in one embodiment receives text and detects that the text includes a URL that refers to audio or audiovisual media content in a catalog of media content, and the messaging app obtains metadata about the media content and transmits the metadata to a second messaging app. Both the first and the second messaging app can operate, through interprocess communication, with its own music extension app that displays a user interface within a view hosted by the respective messaging app.

Inventors: Alsina; Thomas M. (Saratoga, CA), Weinig; Samuel M. (Sunnyvale, CA), Graham; Chance (San Jose, CA), Carrigan; Taylor G. (San Francisco, CA), de Vries; Nathan (San Francisco, CA), Horton; Timothy P. (San Jose, CA)

Applicant: Apple Inc. (Cupertino, CA)

Family ID: 60573212

Appl. No.: 18/989113

Filed: December 20, 2024

Related U.S. Application Data

parent US continuation 18162867 20230201 parent-grant-document US 12218896 child US 18989113

parent US continuation 17305880 20210716 parent-grant-document US 11601385 child US 18162867

parent US continuation 15411176 20170120 parent-grant-document US 11088973 child US 17305880

us-provisional-application US 62349036 20160612

us-provisional-application US 62349101 20160612

Publication Classification

Int. Cl.: **H04L51/046** (20220101); **G06F3/0484** (20220101); **G06F16/48** (20190101); **G06Q30/0601** (20230101); **G06Q30/0645** (20230101); **H04L51/10** (20220101); **H04L51/18** (20220101); **H04L65/61** (20220101); **H04L67/02** (20220101); **H04L67/125** (20220101); **H04L67/565** (20220101); **H04W4/14** (20090101)

U.S. Cl.:

CPC **H04L51/046** (20130101); **G06F3/0484** (20130101); **G06F16/48** (20190101); **G06Q30/0601** (20130101); **G06Q30/0645** (20130101); **H04L51/10** (20130101); **H04L51/18** (20130101); **H04L65/61** (20220501); **H04L67/02** (20130101); **H04L67/125** (20130101); **H04L67/565** (20220501); **H04W4/14** (20130101);

Background/Summary

[0001] This application is a continuation of co-pending U.S. application Ser. No. 18/162,867 filed on Feb. 1, 2023, which is a continuation of U.S. application Ser. No. 17/305,880 filed Jul. 16, 2021, issued as U.S. Pat. No. 11,601,385, which is a continuation of U.S. application Ser. No. 15/411,176 filed Jan. 20, 2017, now issued as U.S. Pat. No. 11,088,973, which claims the benefit of U.S. Provisional Patent Application No. 62/349,036, filed on Jun. 12, 2016, which application is incorporated herein by reference. This application is also related to and claims the benefit of the following U.S. Provisional Patent Application numbers (all of which are incorporated herein by reference): 62/349,101, filed Jun. 12, 2016; 62/349,113, filed Jun. 12, 2016; and 62/349,091, filed Jun. 12, 2016.

BACKGROUND

[0002] The embodiments described in this disclosure relate to messaging systems such as text messaging systems on cellular telephones or other communication devices or data processing systems.

[0003] The use of text messaging systems began many years ago. For example, wireless cellular telephone carriers, such as Verizon or AT&T, allowed text messages through the Short Message Service (SMS) for cell phones in the 1990s before smartphones were available. Typically, the amount of data transmitted has been limited by rules established by the carriers. Recently, as the use of smartphones (e.g. iPhones) and tablet computers (e.g. iPad) has increased, the text messaging systems have developed the ability to send images, such as photos or emojis. In addition, messaging systems such as iMessage from Apple Inc. of Cupertino, California have allowed users to also send and receive text and images through “public” networks which include “public” WiFi access points and the Internet (in addition to using the wireless carrier's private cellular telephone networks), and messaging systems such as iMessage can seamlessly transition between the use of public and private networks depending on the availability of, for example, WiFi access points or the compatibility of the other user's device (which may not be compatible with iMessage).

[0004] A common use of text messaging is the sending and receiving of URLs (Uniform Resource Locators). For example, a user can type a URL or copy and paste a URL into a text message and then send the text message. The URL may be for a web page or a video or a song or some other

content that is accessible, usually through a web browser, to a device. Sending the user entered URL in a text message allows a user at the receiving device to select the URL to cause a web browser (or other component) on the receiving device to load the URL (e.g. a web page) and display or otherwise present content from the URL.

SUMMARY OF THE DESCRIPTION

[0005] One aspect of the embodiments described herein relates to a messaging system which includes, on a first device, a messaging app that is configured to convert a resource locator, such as a URL, that refers to media content in a catalog of media content into metadata which can be transmitted from the first messaging app to a second messaging app on a second device. Another aspect of the embodiments described herein relates to a messaging system which includes, on a client device, a messaging app and at least one media extension app, such as a music extension app, that is configured to present content in a view hosted by the messaging app.

[0006] A method according to one embodiment of the first aspect can include the following operations: receiving, by a first messaging app on a first device, text from a user for delivery to a second device which includes a second messaging app; detecting, with one or more data detectors, a URL (Uniform Resource Locator) in the text, the URL referring to media content in a catalog of media content; transmitting, in response to detecting the URL refers to media content in the catalog, at least a portion of the URL to one or more servers coupled to a source of data about the catalog and configured to respond to URLs from messaging apps with metadata relating to the media content in the catalog; receiving, by the first messaging app, metadata relating to the media content in the catalog; and transmitting the metadata from the first messaging app to the second messaging app, the metadata when received by the second messaging app causing the display of information about the media content in the catalog.

[0007] In one embodiment, the metadata transmitted to the second messaging app can include one or more of: (a) artwork for a song or album or audio visual content; (b) one or more URLs to present a preview of the media content; (c) one or more URLs for playing the media content; or (d) one or more URLs to purchase the media content or subscribe to the media content in the catalog. In one embodiment, the artwork can be displayed within a message bubble in a message transcript displayed by the second messaging app, and the message bubble can display a play command (e.g., a play button). In one embodiment, the metadata can be transmitted to the second messaging app through one or more messaging servers. In one embodiment, the catalog can include content which is one or both of: subscription content or content that can be purchased, and the content can be made available by being streamed from one or more server systems to a device or can be locally stored to a device.

[0008] In one embodiment, the method can also include the further operation of determining, using heuristics, whether to convert the detected URL to the metadata relating to the media content in the catalog. In one embodiment, the heuristics can include at least one rule, such as one or more of: (a) convert the detected URL if the text from the user has only one URL at either the beginning or the end of the text or (b) convert the detected URL if the text from the user contains only one or more URLs and white spaces.

[0009] In one embodiment, the first messaging app and the second messaging app can each be configured to transmit Short Message Service (SMS) text messages and other content and display the text messages in a message transcript, and wherein the text from the user includes non-URL text which is transmitted from the first messaging app to the second messaging app to allow the second messaging app to display the non-URL text along with the message bubble containing the artwork that was transmitted by the first messaging app. In one embodiment, the first messaging app can encrypt the metadata, including the artwork, and also the non-URL text before transmitting the metadata and the non-URL text to the second messaging app through one or more messaging servers.

[0010] In one embodiment, the first messaging app receives a dictionary of metadata from the one

or more servers that are coupled to the source of data about the catalog, and this dictionary of metadata is processed by the first messaging app to derive a link metadata which is a format for a payload transmitted by the messaging app, and the messaging app receives at least a portion of the metadata, such as the artwork, relating to the media content in the catalog after processing the dictionary. In one embodiment, the one or more URLs received by the second messaging app can be communicated through an interprocess communication (IPC) to a media extension app, such as a music extension app, on the second device which can display a user interface of the music extension app in a view of the second messaging app which hosts a view of the user interface of the music extension app.

[0011] A method according to the second aspect in one embodiment can include the following operations: displaying a first messaging app on a first device, the first messaging app configured to communicate text messages with another messaging app such as a second messaging app on a second device through one or more messaging servers; receiving a command to display a first music extension app within a view hosted by the first messaging app, the first music extension app configured to communicate with the first messaging app through an interprocess communication (IPC); displaying, in response to the command, a user interface of the first music extension app within the view, the user interface configured to present media content from a catalog of media content. In one embodiment, the user interface of the first music extension app is configured to display one or more images representing media content either playing or recently played from the catalog of media content. In one embodiment, the user interface of the first music extension app can be configured to receive a selection of the one or more images and, in response to the selection, communicate metadata about the selected media content to the first messaging app through the IPC which in turn sends the metadata to the second messaging app on the second device to share the selected media content when the send command on the second messaging app is selected.

[0012] In one embodiment the metadata can include one or more of: (a) artwork for the selected media content; (b) one or more URLs to present a preview of the selected media content; (c) one or more URLs for playing the selected media content; or (d) one or more URLs to purchase the selected media content or to subscribe to the selected media content in the one or more catalogs.

[0013] In one embodiment, the user interface of the first music extension app can be configured to receive an input to search through one or both of: a locally stored library of media content or a remotely stored list of purchased content or a catalog of subscription media content, and a selected item (e.g., a particular song or album) in the search results obtained from the search can be shared to the second messaging app by a communication through the IPC from the first music extension app to the first messaging app which in turn forwards the metadata relating to the selected item in the search results to the second messaging app through the one or more messaging servers.

[0014] In one embodiment, a command to cause the display of the first music extension app can be one of: (a) a user's selection of an icon in a user interface of the first messaging app; (b) a user's selection of a bubble containing data sent by a second music extension app or the first music extension app; or (c) a selection from a process executing on the first device, the selection from the process performed without user input from the user.

[0015] In one embodiment, the first music extension app can call, through an Application Programming Interface (API) to the first messaging app to request a change of the first music extension app's view within the first messaging app, wherein the change is a switch between a compact view and an expanded view.

[0016] The methods and systems described herein can be implemented by data processing systems, such as one or more smartphones, tablet computers, desktop computers, laptop computers, smart watches, audio accessories, and other data processing systems and other consumer electronic devices. The methods and systems described herein can also be implemented by one or more data processing systems which execute executable computer program instructions, stored in one or more non-transitory machine readable media that cause the one or more data processing systems to

perform the one or more methods described herein when the program instructions are executed. Thus, the embodiments described herein can include methods, data processing systems, and non-transitory machine readable media.

[0017] The above summary does not include an exhaustive list of all embodiments in this disclosure. All systems and methods can be practiced from all suitable combinations of the various aspects and embodiments summarized above, and also those disclosed in the Detailed Description below.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0018] The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings in which like references indicate similar elements.

[0019] The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings in which like references indicate similar elements.

[0020] FIG. 1A shows an example of a user interface for a messaging app on a communication device.

[0021] FIG. 1B shows another example of a user interface for a messaging app on a communication device.

[0022] FIG. 2 shows an example of a messaging system which employs one or more messaging servers to provide a messaging service to a set of client devices.

[0023] FIG. 3A shows a block diagram of an architecture for providing extension apps for use with a messaging app according to one embodiment.

[0024] FIG. 3B shows an example of a user interface of a message bubble created at least in part by an extension app according to one embodiment herein.

[0025] FIG. 3C shows an example of parts of a message bubble based on a template.

[0026] FIG. 4A is a flowchart which shows a method according to one embodiment for browsing, selecting, and launching an extension app from within a messaging app according to one or more embodiments described herein.

[0027] FIG. 4B shows an example of a user interface for a messaging app which includes a browsable view of installed extension apps according to one embodiment described herein.

[0028] FIG. 5A shows an example of a user interface of a messaging app which includes a view of an extension app within the user interface of the messaging app.

[0029] FIG. 5B shows an example of a user interface of an extension app, after the user has created content in the extension app, wherein the view of the extension app is hosted within the user interface of the messaging app.

[0030] FIG. 5C shows an example of a user interface of a messaging app on a communication device after the user has created content using an extension app and has sent the content by using the messaging app to send the content.

[0031] FIG. 5D shows an example of a user interface in a messaging app on a receiving device after the receiving device receives content created by an extension app on the sending device.

[0032] FIG. 5E shows an example of a user interface on a receiving device when the user of the receiving device selects the message bubble containing the content created by the extension app on the sending device so that the user of the receiving device can edit the content on the receiving device.

[0033] FIG. 5F shows an example of a user interface which offers to download and install an extension app for use with the messaging app on the device.

[0034] FIG. 6 is a flowchart which illustrates a method which can be performed on a receiving device according to one embodiment described herein.

[0035] FIG. 7A is a flowchart which depicts a method according to one embodiment in which an extension app can change its view hosted by a messaging app according to one embodiment.

[0036] FIG. 7B shows an example of a user interface of an extension app in an expanded view according to one embodiment.

[0037] FIG. 8 shows an example according to one embodiment in which extension apps on two or more devices can interact with each other through the messaging apps on each device.

[0038] FIG. 9A is a flowchart which illustrates a method according to one or more embodiments described herein for allowing the interaction between extension apps on different devices.

[0039] FIG. 9B shows an example of a message bubble in a user interface which can result from the method shown in FIG. 9A when two or more extension apps interact in a session.

[0040] FIG. 9C shows an example of a message bubble according to one embodiment in which a download and install for a required extension app occurs while the messaging app remains in the foreground.

[0041] FIGS. 9D, 9E, 9F, 9G, and 9H show an example of the message transcripts on two different devices that are involved in a conversation between two messaging apps, one on each device, wherein the conversation involves a session between two extension apps. The user interface shown in FIGS. 9D, 9E, 9F, 9G and 9H show how content in message bubbles created by the extension apps can be converted into breadcrumbs according to one or more embodiments described herein.

[0042] FIG. 10 is a flowchart which illustrates a method according to one embodiment described herein.

[0043] FIG. 11 is a flowchart which illustrates a method according to one embodiment described herein.

[0044] FIG. 12 shows a user interface of a messaging app in which content from two or more different extension apps is presented within the same message transcript provided by the messaging app.

[0045] FIG. 13A is a flowchart which shows a method according to one embodiment for providing backward compatibility for older devices or older messaging systems.

[0046] FIG. 13B shows an example of a user interface in which backward compatibility has been provided within a messaging app.

[0047] FIG. 14A shows a method according to one embodiment in which a receiving device (which has received a message) downloads and installs an extension app required to view or interact with the content in one of the messages.

[0048] FIG. 14B shows an example of a user interface of an extension app marketplace or service from which extension apps can be downloaded and installed.

[0049] FIG. 15A shows an example of a user interface for a messaging app on a communication device after a user has entered a URL in the messaging app.

[0050] FIG. 15B shows an example of the user interface shown in FIG. 15A after the messaging app has converted the URL into an image according to one embodiment.

[0051] FIG. 16 is a flowchart that shows a method according to one embodiment for converting a URL received in a messaging app.

[0052] FIG. 17 shows an example of a set of rules or heuristics which can be used in one or more embodiments to determine whether to convert a URL in a text message.

[0053] FIG. 18 is a block diagram that shows a messaging system on a client device according to one embodiment.

[0054] FIG. 19A shows an example of a user interface for a messaging app after a user has entered a URL in the messaging app, wherein the URL refers to media content in a media catalog.

[0055] FIG. 19B shows an example of a message bubble containing the URL in FIG. 19A after the message has been sent and the URL has been converted.

[0056] FIG. 20A shows an example of a messaging system that interacts with one or more media stores according to one embodiment.

[0057] FIG. **20B** depicts a flowchart that shows a method, in one embodiment, in which a messaging app receives and transmits metadata about a media content item in a catalog of media content, such as songs, albums, movies, books, streaming audio, streaming video, etc.

[0058] FIG. **20C** is a block diagram that shows a messaging system on a client device according to one embodiment.

[0059] FIG. **21A** shows an example of a user interface for a messaging app that hosts a view of a music extension app within the user interface of the messaging app according to one embodiment.

[0060] FIG. **21B** shows an example of the user interface (UI) of the messaging app in FIG. **21A** after the user of the messaging app has selected a recently played item in the music extension app's UI and caused metadata about that item to be sent to another messaging app on one or more receiving devices.

[0061] FIG. **21C** shows a user interface of the another messaging app on one of the receiving devices that received the metadata from the device of FIG. **21B**.

[0062] FIG. **22** shows a flowchart that illustrates a method, in one embodiment, which can be performed on the receiving device of FIG. **21C**.

[0063] FIG. **23** is a table that shows the various possible user interfaces presented on the receiving device of FIG. **21C** in one embodiment.

[0064] FIG. **24** is a block diagram illustrating an exemplary API architecture, which may be used in one or more of the embodiments described herein.

[0065] FIG. **25** shows a block diagram illustrating a software stack which can include one or more applications and services in operating systems.

[0066] FIG. **26** shows an example of a data processing system which can be used in one or more embodiments described herein.

DETAILED DESCRIPTION

[0067] Various embodiments and aspects will be described with reference to details discussed below, and the accompanying drawings will illustrate the various embodiments. The following description and drawings are illustrative and are not to be construed as limiting. Numerous specific details are described to provide a thorough understanding of various embodiments. However, in certain instances, well-known or conventional details are not described in order to provide a concise discussion of embodiments.

[0068] Reference in the specification to “one embodiment” or “an embodiment” means that a particular feature, structure, or characteristic described in conjunction with the embodiment can be included in at least one embodiment. The appearances of the phrase “in one embodiment” in various places in the specification do not necessarily all refer to the same embodiment. The processes depicted in the figures that follow are performed by processing logic that comprises hardware (e.g. circuitry, dedicated logic, etc.), software, or a combination of both. Although the processes are described below in terms of some sequential operations, it should be appreciated that some of the operations described may be performed in a different order. Moreover, some operations may be performed in parallel rather than sequentially.

[0069] The various embodiments described herein relate to messaging systems such as text messaging systems or “chat” messaging systems or other systems which allow devices to communicate messages between the devices. For example, iMessage from Apple Inc. of Cupertino, California is an example of a messaging service for iOS devices and Mac (OS X) computers. Typically, a messaging system includes the plurality of client devices, each including at least one messaging app, and a set of one or more messaging servers that can receive messages from client devices and transmit messages to client devices. FIG. **1A** shows an example of a user interface of a messaging app on a client device. The client device can be a communication device **10** which can be a smartphone, or tablet computer, or a desktop computer or a laptop computer, wearable, on-board computer, or other data processing systems or other consumer electronics devices. In one embodiment, the device can include a conventional touch screen that can both display images and

also accept touch inputs from a user. The touch screen **12** on the communication device can display the user interface of the messaging app which can include a message transcript **16** and an on-screen keyboard **20** below the message transcript **16**. In addition, the user interface of the messaging app can include a user name **14** indicating the recipient, in one embodiment, of messages sent from the communication device **10**. In addition, the user interface can include a text entry region **18** which indicates the content of the text entered by the user before it is sent; in a sense, the text entry region **18** is a text staging area indicating text that is ready to be sent to the recipient.

[0070] FIG. **1B** shows a more detailed example of a user interface of a messaging app on a communication device **10A**. The user interface is displayed in this embodiment on touch screen **12A** and includes on-screen keyboard **20A**, text entry region **18A**, a message transcript **16A** and a user name **14A** showing the name of the other user(s) to whom messages are sent and from whom messages are received. The text entry region **18A** is a staging area for content such as one or more of text, stickers, extension app content, images etc. which are ready to be sent in response to the user's selection of the send command (and in one embodiment, the content can be edited in the staging area). In the example shown in FIG. **1B**, messages from Freddy (user name **14A**) are shown on the left side of the message transcript **16A** and messages sent by the user of communication device **10A** are shown on the right side of message transcript **16A**. Hence, message bubble **17** shows the message "Where at?" sent by the user of communication device **10A** to Freddy as a response to the message from Freddy "Dinner tonight?". The message in message bubble **17** causes Freddy to respond with "How's this?" shown in message bubble **19**. The use of the word "bubble", such as message bubble or session bubble, etc. is not meant to imply any specific shape or form; rather it is intended to mean any shape or form of demarcation between messages between two or more participants, and thus the demarcation can use boxes or lines or message containers or different colors, etc. Thus the phrase "message bubble" is meant to cover all such demarcations (or other ways to distinguish) between messages between two or more participants, and particularly in the context of such demarcations or other ways to distinguish in a message transcript. In one embodiment, the message transcript can be scrolled up and down and messages in the message transcript are presented in sequence according to their time, thus the user can see the actual chat or conversation over time by scrolling the view up or down. The user interface shown in FIG. **1B** also includes three icons **22**, **23**, and **24** above the on-screen keyboard **20A** and to the left of the text entry region **18A**. The extension apps view icon **22**, when selected, causes the display of a view of the installed extensions app that can operate with the messaging app, and the view provided can be a browsable view, such as the browsable view **157** shown in FIG. **4B** to allow the user to scroll through multiple pages showing all of the installed extension apps that are configured to operate with the messaging app according to the one or more embodiments described herein. The imaging app icon **23** can in one embodiment be an icon that when selected causes the launching of a plugin of the messaging app that provides image creation within the messaging app's process such as the plugin shown in FIGS. **5A**, **5B**, and **5C**. The camera app icon **24** in one embodiment can, when selected, cause the communication device **10A** to enter into a camera mode in which a camera of a device can capture still images or video images which can be placed into the message in order to send the image or video.

[0071] A brief overview of an example of a messaging system will now be provided in conjunction with FIG. **2**. A messaging system **50** can include a plurality of client devices, such as client devices **53** and **54**. Each of these client devices can include at least one messaging app that is configured to operate with extension apps according to the one or more embodiments described herein and also communicate at least text messages and optionally resource locators or images or other content (e.g. as described relative to FIG. **13A**) for devices that are not compatible with the extension app architecture in a messaging app. In a typical messaging system, there can be millions of client devices communicating through a set of messaging servers. In one embodiment, a plurality of messaging servers can be configured to receive encrypted messages from sending devices and then

transmit those encrypted messages to the receiving devices. Another set of servers can be configured to receive non-text content, such as images or other “attachments” and provide those images or attachments in download operations to receiving devices in response to requests from those receiving devices to obtain the images or attachments. In one embodiment, a sender's outgoing message is individually encrypted for each of the receiver's devices. In one embodiment, an asymmetric RSA encryption algorithm can be used to perform the encryption. In one embodiment, the public RSA encryption keys of each of the receiving devices can be retrieved from a directory service (maintained by the one or more messaging servers) which includes a database, such as a database 52 which is coupled to the one or more messaging servers 51. When a client device, such as client device 53 seeks to send a message to another client device, it identifies the other client device (such as by an email address or a telephone number or other identifier) to the one or more messaging servers 51. That identifier is sent from the client device, such as client device 53 to one or more messaging servers 51 which then perform a lookup operation in the database 52 based upon the supplied identifier to retrieve the public key that corresponds to that identifier. That public key is then transmitted back to the client device which requested the public key for that particular receiving device, and then the client device can encrypt the message using the public key or using another key (e.g. a symmetric key) which can be randomly generated, and that other key is encrypted with the public RSA encryption key for the particular receiving device. In one embodiment, the randomly generated key can be randomly generated on a per message basis. In one embodiment, the resulting messages, one for each receiving device, consists of the encrypted message text, the encrypted message key, and the sender's digital signature, and this resulting message for each receiving device is then uploaded to the one or more messaging servers 51 for delivery to the recipient client devices, such as client device 54. In one embodiment, the messaging system 50 can be configured to operate through “public” networks which include public WiFi access points (such as WiFi access points in coffee shops, airports, etc.) and also the Internet. The messaging apps on each of the client devices 53 and 54 can also be configured to operate with the “private” networks provided by wireless cellular telephone carriers, such as Verizon and AT&T, and the messaging apps can be configured to seamlessly switch between the use of the private and public networks depending upon the availability of each and also depending upon the compatibility of each of the client devices in a messaging session. In one embodiment, the messaging servers 51 can include a set of push notification servers which receive the uploaded text messages and which “push” those text messages to receiving devices.

[0072] A messaging system in one embodiment on a client device includes a messaging app and one or more extension apps that each operate as separate processes. In one embodiment, the message app and the one or more extension apps can each be separate sandboxed processes that operate or execute in their own memory spaces. In addition, the messaging app can also operate with plug-ins, such as an image creation plug-in shown in FIG. 5A, which operate within the same process and memory space as the messaging app. The messaging app and each extension app communicate with each other through an interprocess communication, such as the XPC framework provided in iOS and Mac OS X. The messaging app is designed to receive text on a sending device from a user of the device and display the text in a message transcript and send the text, through a set of one or more messaging servers, to a receiving device which displays, through a counterpart messaging application on the receiving device, the received text in a message transcript on the receiving device. The receiving device and the sending device can each have a copy of the same extension app which are configured to create content of a certain type depending upon the particular extension app (or, in an alternative embodiment, each can include a copy of different extension apps that are compatible for the content used by them).

[0073] FIG. 3A shows an example of a software architecture in which a messaging app and one or more extension apps operate together to provide an enhanced messaging system. As shown in FIG. 3A, the messaging system 75 includes a messaging app 76 and a set of plug-in modules, such as a

composition module **77** and a data transport module **79** which are configured to communicate through an interprocess communication (IPC) **81** with the one or more extension apps **83**. As shown in FIG. **3A**, the messaging app and the composition module **77** and the data transport module **79** operate within the messaging app processes in a memory space controlled by a kernel on the communication device which executes the messaging system **75**. The composition module **77** composes the content of a message bubble when the message bubble displays or otherwise presents content passed to the messaging app through the IPC **81**. The data transport module **79** passes the content and other data to the extension app through the IPC **81** and receives the content and other data from the extension app through the IPC **81**. Both modules **77** and **79** in one embodiment can have an extendable plug-in architecture that allows the addition of new plug-ins for new extension apps that generate new content or require a new data transport process. In this context, the plug-ins are additional software that operate within the same process as the messaging app. The composition module **77** can use a template to construct a message bubble such as the “MSMessageTemplateLayout” described below relative to FIG. **3C**. The kernel can include a software library or software framework for providing the IPC **81** to allow communication between the messaging system **75** and the one or more extension apps **83**. In one embodiment, the IPC framework can include a system area that is referred to as an extension point which provides APIs to allow for communication between the two different processes and to reinforce policies in terms of the types of communications which are permissible. In one embodiment, the communication through IPC involves the placement (writing) of content in a memory region by one process and the IPC framework allows another process to read from that memory region. In one embodiment, the messaging app **76** can launch automatically the extension app processes and can manage their lifetime, including the termination of those processes. In one embodiment, each of the extension apps **83** run in their own address space communication between the extension and the messaging app uses the IPC mediated by the system framework and they do not have access to each other's files or memory spaces. In one embodiment, each of the extension apps may be a sandboxed process which are separate from each other and the messaging app **76** can also be a separate sandboxed process which is separate from the sandboxed processes of the extension apps. Moreover, the extension apps can be provided with fewer system permissions relative to the messaging app so that the extension apps operate in a more restricted environment than the messaging app. Further information in connection with the use of interprocess communications for extensions can be found in U.S. patent application Ser. No. 14/488,122, filed Sep. 16, 2014 and published as U.S. Publication No. U.S. 2015/0347748, which application is incorporated herein by reference.

[0074] In one embodiment the messaging app provides a view of content obtained from the extension app through the interprocess communication. The extension app can create the content in its own process and then provide that content in formats known to be acceptable to the messaging app (such as standard image formats or other standard formats). This allows the messaging app to then present the content from the extension app within one or more message bubbles within a message transcript (without needing to execute the extension app at least on the receiving device). FIG. **3B** shows an example of a message bubble **17A** which contains content created by and provided from an extension app, shown as content **85** which also can include a text message created by or provided by the extension app, such as text message **86**. In one embodiment, the message bubble **17A** can also include an icon **87** which can be the icon of the extension app which created the content **85**.

[0075] Objects created by an extension app in one embodiment are shown in the message transcript on sending and receiving devices without launching the extension app. The extension app should provide enough information to construct a message bubble as part of the object. The object can consist of some opaque data encoded in a resource locator and a layout specification provided as a MSMessageTemplateLayout object.

[0076] `MSMessage TemplateLayout` is a subclass of `MSMessageLayout` and represents one method of specifying message bubble layout.

[0077] `MSMessage TemplateLayout` can have the following properties in one embodiment which are shown in FIG. 3C: [0078] 1) `image` or `mediaFileURL`: An image provide as a `UIImage` or as a file URL to an image file or a file URL to a video [0079] 2) `imageTitle`: A string the will be rendered on top of the image or movie [0080] 3) `imageSubTitle`: A string the will be rendered on top of the image or movie below the image Title [0081] 4) `caption`: A string the will be rendered in a caption bar below the image or movie [0082] 5) `trailingCaption`: A string the will be rendered right aligned in a caption bar below the image or movie [0083] 6) `subCaption`: A string the will be rendered in a caption bar below the caption [0084] 7) `trailingSubCaption`: A string the will be rendered right aligned in a caption bar below the trailingCaption [0085] 8) `Extension icon`: This is not provided as part of the `MSMessageTemplateLayout` but is derived from the bundle identifier of the extension that created the `MSMessage`.

[0086] The messaging app can use this information to construct the message bubble similar to the example shown in FIG. 3C.

[0087] The `MSMessage TemplateLayout` is serialized and transferred to the remote devices along with the opaque data. On receipt the messaging app on the receiving device will create a `MSMessage TemplateLayout` using the serialized data and use this to draw the message bubble in the receiver's message transcript.

[0088] In one embodiment, the extension apps which are configured to operate with a messaging app are not executable outside of the messaging application, and thus their life cycle is managed entirely by the messaging app. Moreover, as further described below, the downloading and installing of the extension apps can be controlled exclusively by the messaging app in one embodiment.

[0089] In one embodiment, each extension app can be obtained from an app marketplace or distribution facility such as the Apple App Store (trademark) for message extension apps and can be launched from within the messaging app. FIG. 4A shows an example of a method according to one embodiment in which the installed extension apps can be browsed from within the messaging app, and a particular extension app can be launched to allow the user to interact with a particular extension app. The method can begin in operation **101** in which the messaging app displays its message transcript and an on-screen keyboard. FIG. 1B shows an example of a user interface of such a messaging app. Then in operation **103**, the messaging app can receive an input to display a browsable view of installed extension apps within a view of the messaging app. For example, a user may tap on the icon **22** (in FIG. 1B) to select the icon which in turn causes the display of the browsable view in operation **105**. In one embodiment, the browsable view of installed extension apps takes the place of the on-screen keyboard and retrieves the list of the installed extension apps from an extension registry which causes the display of icons for each of the installed extension apps. FIG. 4B shows an example of the result of operation **105** in which a browsable view **157** is displayed and takes the place of the on-screen keyboard of the messaging app as shown in FIG. 4B. [0090] Referring to FIG. 4B, it can be seen that the browsable view **157** includes a plurality of icons each representing one of the installed extension apps that can operate with the messaging app that provides the user interface shown in FIG. 4B. The user interface of the messaging app shown in FIG. 4B includes a message transcript **153** shown in the upper portion of the touch screen **151** on the communication device **150** which executes the messaging app. A user name **155** of the other user (recipient of messages sent from communication device **150**) is shown at the top of the user interface of the messaging app. A text entry region **155** (which shows text or other content that is staged or ready to be sent) is displayed between the message transcript **153** and the browsable view **157**. In one embodiment, the browsable view is browsable by swiping the user's finger across the touch screen to cause the pages in the various views of installed extension apps to be displayed (and in one embodiment, can also show, for example, uninstalled extension apps that either need to

complete a download process or need to complete an install process). In one embodiment, a page indicator **159** at the bottom of the user interface can show the current page of installed extension apps. In one embodiment, one of the icons can be reserved to launch or enter the extension app marketplace, an example of which is shown in FIG. **14B**. In another embodiment the extension app marketplace can be displayed in the browsable view **157** in response to selecting icon **167**. In the embodiment shown in FIG. **4B**, the icon **167** is the extension app marketplace icon which can be selected to cause the presentation of the extension app marketplace, such as the extension app marketplace shown in FIG. **14B**. In the embodiment shown in FIG. **4B**, the icon **169** can be selected (e.g. user taps icon **169**) to cause the display in the browsable view of recently sent stickers or handwritten messages or other recently sent items or recently used apps, etc. The display of recently sent items, etc. can in one embodiment be provided by a plug-in operating within the messaging app's process. Other extension apps include a restaurant reservation app icon **161** which can be an extension app that can be used to engage a restaurant reservation service, such as Open Table. Another example of an extension app is represented by icon **163** which when selected launches a restaurant review app which provides reviews of restaurants and which can be similar to the reviews provided by Yelp, for example. The other extension app icons **165** and **171** represent other extension apps which are installed and which can be launched by selecting one of those extension app icons.

[0091] Referring back to FIG. **4A**, once the browsable view is displayed by operation **105**, the user can select one of the extension apps by selecting the corresponding icon which in turn causes the launching of the selected extension app in operation **107**. In one embodiment, the messaging app calls a system service to launch the selected extension app and prepare it for execution as an extension of the messaging app using the architecture shown, for example, in FIG. **3A**. Once the selected extension app has been launched and is executing, the messaging app, such as messaging app **76** can host the view of the content provided by the executing extension app through the IPC framework as described herein. For example, in operation **109** shown in FIG. **4A**, the messaging app can display a view of the content provided by the extension app within a portion of the view of the messaging app. FIGS. **5A** through **5F** will now be described as examples of how the messaging app hosts the view of the content of the executing extension app.

[0092] FIG. **5A** shows an example of a messaging app that hosts a view of a plug-in of a messaging app that is an image creation app launched by selecting the icon **207** (e.g. icon **207** is touched or tapped or otherwise selected by a user). In another embodiment, the plug-in shown in the bottom portion of the user interface shown in FIG. **5A** may be launched by selecting one of the icons in the browsable view **157** shown in FIG. **4B**. While the example shown in FIG. **5A** can be implemented as a plug-in of the messaging app, in another embodiment, the example shown in FIG. **5A** can be an extension app. In the example shown in FIG. **5A**, the view of the plug-in (or extension app) has taken the place of the on-screen keyboard of the messaging app, but the message transcript of the messaging app is still viewable and displayed in the user interface, allowing the user to scroll through the message transcript in one embodiment to see the entire transcript. In another embodiment, the view of the plug-in or extension app is an overlay on top of the on-screen keyboard, a portion of which may be viewable. The transcript is displayed on the touchscreen **202** in one embodiment on the communication device **200**. The user interface of the message app also includes the user name **203** which presents in one embodiment the name of the other user in the conversation or chat. The user interface of the message app also includes the text entry region **211** which is similar to the text entry region **18A** and the text entry region **155**. The plug-in (or extension app) includes a drawing canvas **215** and various controls and options which can be selected by the user to draw or create images. Drawing controls **212** in one embodiment can allow the users to select different colors to sketch on the drawing canvas if the sketch option **217** is selected. The plug-in (or extension app) can also provide taps as a message if the tap option **221** is selected. The plug-in (or extension app) can also provide heartbeats if the heartbeat option **219** is

selected. In one embodiment, the sketch, heartbeat, and taps can be similar to the sketch, heartbeat and taps on the Digital Touch application which is available on the Apple Watch. The plug-in (or extension app) also includes an expanded view icon **223** which when selected can cause the plug-in (or extension app) to switch from its current compact view shown in FIG. 5A to an expanded view, such as the expanded view shown in FIG. 7B. In the example shown in FIG. 5A, the user has just launched the plug-in image creation app (or an extension app in the alternative embodiment) and has not yet created any content. This can be contrasted with the state of the plug-in (or extension app) shown in FIG. 5B in which the user has created a happy face sketch by using the sketch option **217** and by drawing on the drawing canvas **215** with, for example, the user's finger. The user can then cause the drawing to be sent to the other user (or users) who would be recipients for the message. Thus for example, the user of the device **200** can select a send command from within the messaging app executing on communication device **200** to cause the content created by the plug-in (or extension app) to be sent to the recipients. The result of the sending operation is shown in FIG. 5C in which the happy face has been sent to Freddy as indicated by the user name **203**. The message bubble **230** shows the happy face sketch that was created by the plug-in (or extension app); in the case of the extension app, this created content was communicated from the extension app through the IPC frame work to the messaging app and then presented within the message bubble **230** to show that the message containing that content was transmitted to one or more recipients. In one embodiment, a delivery indicator **231** can indicate that the message was delivered and a keep control **232** can be provided to the user to allow the user to retain the content in the message transcript in those embodiments in which the content can be automatically eliminated after a period of time.

[0093] FIGS. 5D, 5E, and 5F illustrate what can happen on the receiver's device in response to receiving the content from the communication device **200** shown in FIG. 5C. Also, the flowchart shown in FIG. 6 can show the method performed at the receiving device, such as the communication device **250** shown in FIGS. 5D, 5E, and 5F. Referring now to FIG. 5D, it can be seen that the communication device **250** (used by Freddy) has received the happy face content which is shown in message bubble **253**. This content was created by the extension app executing on communication device **200** which provided the happy face content to the messaging app executing on communication device **200** which in turn transmitted that content through a messaging service (e.g., a set of messaging servers, such as the messaging servers **51** shown in FIG. 2) to the messaging app executing on the communication device **250** which in turn presented the content in the message bubble **253**. In one embodiment, the content can be rendered by the messaging app as it uses known standard formats (including standard image, audio, and video formats in one embodiment) and thus the extension app need not be installed or executing in order to show the content created by the counterpart (remote) extension application on the sending device. Thus in this case as shown in FIG. 5D, the message bubble **253** can present the content even though the counterpart extension application may not be executed or may not even be installed on the communication device **250**. The messaging app shown in FIG. 5D includes, in its user interface, a message transcript **201A**, a text entry region **211**, and an on-screen keyboard **255**. In one embodiment, content received from a remote extension app will not cause the launching automatically of the corresponding extension app on the receiving device even if the corresponding extension app is installed. In this embodiment, the corresponding extension app on the receiving device can be launched by the user's selection of the message bubble containing the content created by the remote extension app. If the user does select that content by, for example, touching or otherwise selecting the message bubble **253**, then the extension app that corresponds to the remote extension app will be launched if installed on the communication device **250**. The result is shown in FIG. 5E in which the user interface of the extension app has occupied the space in which the on-screen keyboard **255** was previously displayed and shows the happy face drawing within the canvas **215** allowing the user of the communication device **250** to alter or otherwise modify that sketch

and perhaps send it back to the other user in the chat session or conversation. If, on the other hand, the corresponding extension app is not installed on the communication device, then in one embodiment the messaging app can present a notice to the user asking or offering to install the app for the particular message bubble which was selected. An example of this is shown in FIG. 5F in which the notice **259** includes two user selectable options, one of which will install the required app for the message bubble. In another embodiment, a sheet that shows a page of information from an extension app marketplace can be displayed within the view of the messaging app.

[0094] In one embodiment, the message transmitted from the remote device to the communication device **250** contains metadata which specifies the remote extension app used to create the content. In one embodiment, this metadata can be an app identifier, such as an identifier provided by an app marketplace or an extension app marketplace from which the extension apps can be downloaded and installed or can be a different identifier that can be associated with the identifier used by the app marketplace. In one embodiment, the notice **259** can result from the selection of the message bubble **253**, while in another embodiment it can result automatically if the app identifier in the metadata for the content is not installed when the content for the message bubble **253** is received by the communication device **250**.

[0095] FIG. **6** will now be referred to in conjunction with FIGS. **5D**, **5E**, and **5F** to explain a method in one embodiment in which a receiving device processes content created by a remote extension app, such as the remote extension app executing in conjunction with the messaging app on communication device **200**. In operation **301**, a communication device can receive a message with content created by an extension app, such as a remote extension app operating in conjunction with a messaging app on a sending device. In addition, the communication device can also receive the metadata which can include a message bubble identifier, session identifier and an extension app identifier and potentially other data such as an optional resource locator and other data (state information) that may be associated with the optional resource locator and the resource locator may also include state information that is encoded into the resource locator. Further information with respect to the use of resource locators and data associated with the resource locators will be provided in conjunction with FIGS. **8**, **9A**, and **9B**. Then in operation **303**, the communication device which has received the message in operation **301** processes the content and displays the content in a message bubble which was identified by the message bubble identifier, and the message bubble is displayed within the message transcript. In one embodiment the processing of the content can include decrypting the content and rendering the content for presentation and display within the message bubble. In one embodiment, the content is displayed by the messaging app without assistance from the extension app; in other words, the extension app may not be installed or if installed is not executing on the communication device, and hence the on-screen keyboard is displayed in one embodiment. Then in operation **305** the communication device receives a selection of the message bubble displaying the content created by the remote extension app. In one embodiment, referring to FIG. **5D**, the user can tap on the message bubble on a touchscreen or otherwise select (e.g. user uses a stylus on a touchscreen or uses a mouse with a desktop computer, etc.) the message bubble **253** to cause the selection in operation **305**. In response to operation **305**, the communication device, such as communication device **250** determines in operation **307** whether or not the extension app is installed. This can be performed in one embodiment by examining a list or registration of installed extension apps maintained by the messaging app. In one embodiment, the metadata received in operation **301** includes the app identifier, and the messaging app in operation **307** searches the list to determine whether the identifier is present in the list. If the identifier is not present in the list then the messaging app determines that the extension app is not installed, thereby causing the communication device to perform operation **309** in which the notice **259**, shown in FIG. **5F**, can be displayed to the user to offer to download and install the app specified by the app identifier which was received as part of the metadata in operation **301**. If the user selects the option “yes” then operation **311** is performed

in which the messaging app causes the communication device to access the extension app marketplace (such as the extension app marketplace having a user interface shown in FIG. 14B) to retrieve through a download process a copy of the extension app and to cause the extension app to be installed. In one embodiment, operation 311 may be performed entirely in the background so that the messaging app remains the foreground app during the download and install process. FIG. 9C shows an example of a message bubble 471 maintained in a message transcript during the download and install process, where the message bubble includes a progress bar 473 which indicates the progress of the download and install operation while the messaging app remains the foreground application displaying the message bubble 471 in the message transcript of the messaging app. In another embodiment of the operation 311, a sheet that shows a page of information from an extension app marketplace can be displayed over the messaging app (with a portion of the messaging app optionally still displayed), and the sheet can show a “buy” or install or download button, which if selected, can cause the download and installation of the extension app, and the sheet can be dismissed (removed from display) either by selecting a dismiss command or by selecting buy or install or download. After downloading and installing the extension app, processing can proceed to operation 313 in FIG. 6 in which the extension app is launched and the content and other data used or created by the remote extension app is passed (via IPC) to the extension app by the messaging app, and the extension app is displayed in either a compact or expanded view in one embodiment and the content created by the remote extension app is displayed within that view. As shown in FIG. 6, operation 313 also follows operation 307 if operation 307 determines that the extension app is installed. FIG. 5E shows one example of the result of operation 313.

[0096] In one embodiment, a messaging app can launch different types of extension apps in different ways depending upon the type of the extension app. For example, one type of extension app can be launched automatically in response to receipt of a message bubble containing content from an extension app having a certain predetermined type. Other extension apps having a different type may only be launched in one embodiment in response to the selection of a message bubble containing content from that extension app or by the selection of an icon representing the extension app in a browsable view, such as browsable view 571. It may be desirable to allow certain extension apps having a certain type to be launched automatically in response to receiving content that is displayed within the message transcript while not automatically launching other types of extension apps. In another alternative embodiment, one or more extension apps can be permitted to execute in the background and can be allowed to update their respective user interfaces that are presented in their respective message bubbles.

[0097] In an alternative embodiment, the metadata can include a format or extension identifier such as an identifier of an image format that can be used to determine available extension apps that can process that image format on the receiving device.

[0098] FIGS. 7A and 7B illustrate another aspect of the embodiments described herein in which the extension app can cause a view to be changed by sending communications to the messaging app to cause that change. In one embodiment, an application programming interface (API) can be provided between the extension app and the messaging app to allow the extension app to make calls to that API to change its view within the messaging app which hosts the view of the extension app. In one embodiment, an extension app can have at least two different views which can include a compact view and an expanded view. In one embodiment, the compact view may be a view which takes the place of the on-screen keyboard of the messaging app while the message transcript remains displayed in the user interface of the messaging app. In the expanded view, the message transcript is no longer displayed and the on-screen keyboard is not displayed but certain other components of the user interface of the messaging app, such as text entry region 211 and a camera activation icon 235 are displayed. FIG. 7B shows an example of an expanded view in which the drawing canvas 215A occupies most of the space of the touch screen. As shown in FIG. 7B, the

compact view icon **223A** can be selected by the user to cause the system to change from the expanded view shown in FIG. 7B back to the compact view, such as the view shown in FIG. 5A.

[0099] The method shown in FIG. 7A is one embodiment showing how the view can change, and it will be appreciated that in alternative embodiments, the sequence of operations may be performed in a different order and there may be omitted steps or intervening steps or additional steps.

[0100] In operation **351** of FIG. 7A, the extension app is displayed by the messaging app with a particular view or style. In operation **353**, the extension app can make a call to the messaging app to get the current presentation view/style of the extension app. In operation **357**, the messaging app can provide a return for the call from operation **353**, and that return can indicate the current presentation view/style of the extension app. In response to that current presentation received in operation **357**, the extension app can request a change in the presentation view/style by providing a call to the messaging app to cause that change to occur, and that call is received in operation **361**. The messaging app in one embodiment can initially respond to this call with an acknowledgement that the view is being changed or will be changed. In response to the call, in operation **363**, the messaging app changes the presentation style/view and displays the extension app within the requested view presentation view/style and in operation **365** the messaging app informs the extension app that the change of view has been completed. Referring back to FIG. 7B, if the user selects the compact view icon **223A**, this can cause a call from the extension app to the messaging app to change the view of the extension app within the user interface of the messaging app.

[0101] FIGS. 8, 9A, 9B, and 9C will now be described relative to another aspect of the embodiments presented herein. In one aspect of this embodiment, an extension app and a counterpart extension app on another device can engage in a communication session and exchange information back and forth in their communication session, and all of this occurs within the context of a message transcript maintained by the two messaging apps which interface between the two extension apps as shown in FIG. 8. In one embodiment, plug-ins for a messaging app can also operate in a similar way and exchange information between the plug-ins through two messaging apps that are in a session. The messaging system **400** shown in FIG. 8 includes at least two clients devices, client device **401** and client device **405** and also includes a set of one or more messaging servers **403**. The client devices **401** and **405** can be similar to the client devices **53** and **54** in FIG. 2, and the set of one or more messaging servers **403** can be similar to the set of messaging servers **51** shown in FIG. 2. Each client device can include an installed copy of a particular extension app (such as, for example, an extension app for making reservations at restaurants), and the extension app on each device can be used to create content (e.g. text, images, audio, video, etc.), and this content is passed, through an interprocess communication framework to the messaging app on the device for a specific message, which can be referred to as a message bubble, in the message transcript on the particular client device. The message app receives the content (and optionally other data from the extension app, including for example an identifier of the extension app, a resource locator and optionally metadata for use by a counterpart or remote extension app on the other device, etc.) and displays the displayable content (such as an image of a restaurant provided by the extension app which is a restaurant reservation app such as “Open Table” in the message bubble in the message transcript). In effect, the messaging app hosts a view within the messaging app and the content of that view is provided by the extension app. The resource locator and metadata in one embodiment are opaque to the messaging app (e.g. cannot be recognized by the messaging app) but can be used by the extension apps on each device to maintain state information for the session between the extension apps, and the messaging apps on each device act as a communication mechanism between the extension apps by passing the resource locator and metadata between the extension apps. In one embodiment, state information about the session may be encoded in the resource locator or may be provided in the metadata or may in both. In one embodiment, the content created from a session on each device is displayed within the same single message bubble (identified by a session identifier that can be maintained by the messaging app) in

the message transcripts, and each time the content changes (based on changes from either device), the updated content continues to be displayed within a single message bubble in the message transcript and any prior message bubbles that displayed content in the session can be converted into breadcrumbs, and these prior message bubbles will also include the same session identifier as the new, updated message bubble. The operations and functions of the components shown in FIG. 8 will now be described while referring to FIG. 9A which shows a method of operating the messaging system **400** in one embodiment.

[0102] In operation **451** of FIG. 9A an extension app, such as the extension app **407** can create content and generate a resource locator and data (or can modify existing content, resource locator or data). The extension app can be similar to the extension app **83** and is executing in one embodiment in one process while the messaging app, which can be similar to the messaging app **76**, is executing in another process and communication between the processes occurs through an IPC, such as the IPC **81** which can be a software framework or library for providing interprocess communication between two distinct processes. The extension app **407** can be, for example, a restaurant reservation app that creates a reservation through a website that can provide a state information for use with the resource locator (or the state information can be encoded into the resource locator). In one embodiment, the data provided with the resource locator can be a state information provided from the website, and the state information can include information about the particular restaurant and the time of the reservation and the number of people in the reservation. The extension app **407** can present, in either a compact view or expanded view the user interface for making the restaurant reservation through the website while the remainder of the messaging app is displayed, including for example the message transcript. Thus the user of the extension app **407** can see the context of the conversation in the message transcript while interacting with the extension app and the website (through the extension app) to create a restaurant reservation. In one embodiment, the user may be able to browse through various restaurants (within the restaurant reservation app) and search for restaurants. After the user of the extension app **407** has selected a restaurant and entered a reservation, the extension app **407** can pass, through the IPC **415**, content and also the resource locator and data **417** shown in FIG. 8 for the messaging app on device **401**. This is shown as operation **453** in FIG. 9A. In operation **455**, the messaging app **409** associates the content received from the extension app **407** with the message bubble and displays content in the bubble in the message transcript in the user interface of the messaging app **409**. Then in operation **457**, the messaging app **409**, in response to a send command received from the user, sends a message (if any) and content received from the extension app **407**, and an app identifier which identifies the app **407** (and optionally an icon of the app **407**), and the resource locator and the data (if any) and a session identifier to the second device through the one or more messaging servers **403** which pass the communication **419** to the second device **405** (also referred to as client device **405**). In one embodiment, the operations **453** and **455** can occur as part of operation **457** in response to the messaging app receiving the selection of the send command. The icon of the app **407** can be displayed on the message bubble on the receiving device even if the counterpart extension app is not installed; see for example, icon **87** in FIG. 3B. In operation **459** shown in FIG. 9A, the messaging app **411** on the client device **405** receives the content from the one or more messaging servers **403** and displays the content in the identified message bubble **421** within the message transcript provided by the user interface of the messaging app **411**. FIG. 9B shows a more detailed example of such a message bubble **471** having content created by a restaurant reservation extension app in one embodiment. In the example shown in FIG. 9B, the content includes the name of the restaurant, the time of the reservation and the number of persons of the reservation. In one embodiment, this content can be displayed by the messaging app without requiring that the extension app **413** be launched. In one embodiment, the extension app **413** is not launched until the user of client device **405** selects the message bubble **421**, thereby indicating to the client device that the user of the client device **405** intends to interact with the content in the message bubble **421**. In

an alternative embodiment, the extension app **413** can be launched upon receipt by the messaging app **411** of the content but remain in the background and be ready for execution when the user of client device **405** enters a command to cause the extension app to appear. In operation **461**, the extension app **413**, which is identified by the app identifier provided in the communication **419** is launched, if not already launched, in response to selection of message bubble **421** and the messaging app **411** passes, through IPC **423** the content associated with message bubble **421** and resource locator and data **425** to the messaging app **413**. In one embodiment, the extension app **413** is the counterpart extension app that is the same extension app as extension app **407**, while in another embodiment they can be merely compatible in that they can process the same type of content.

[0103] At this point, the extension app **413** can receive user input by the user of client device **405** and can modify one or more of the content, the resource locator or the data. For example, the user of client device **405** can cause the extension app **413** to access one or more websites to make a modified restaurant reservation by modifying the time, the number of people, the particular restaurant, etc. In one embodiment, the extension app **413**, and also the extension app **407** can interact directly (but separately and independently) with the web server by sending the resource locator and the data to the web server and receiving responses from the web server which may include modified data or modified resource locators, or new data and/or new resource locators etc. In one embodiment, the web server can store data for use during the session, and this stored data can include information for some or all of the state information that can also be maintained by the two extension apps in the session. Again, if the extension app **413** is presented for display in a compact view, then the user of the device **405** can interact with the extension app **413** to make the restaurant reservation while the context and conversation of the chat or messaging session is shown in the message transcript of the messaging app **411**. The user of the client device **405** can scroll through the message transcript while continuing to look at and interact with the extension app **413**. Thus, the extension app **413** can, in operation **463** receive user input and may modify at least one of the content, the resource locator or data, and then can pass, in operation **465** the resource locator and data **427** (which may be modified or new) to the messaging app **411**. In turn, the messaging app **411** in operation **467** can send the content, which may be modified, and the app identifier and the resource locator (which may be modified) and data (which may be modified) and the bubble ID back to the client device **401**. As shown in operation **469**, this process can repeat over time as the two users work on setting up a restaurant reservation in the example provided herein.

[0104] It will be appreciated that many different types of extension apps can provide a collaborative environment between the users of client devices **401** and **405** to exchange information and collaborate together and that restaurant reservation is one such type. Thus, it will be appreciated that the restaurant reservation example that is described relative to FIGS. **8** and **9A** is merely an example of one such type of extension app which can provide a collaborative environment in the context of the user interface of a messaging app. Examples of other types of extension apps which can provide a similar collaborative environment include, for example: sticker extension apps; imaging apps; drawing apps; content creation apps; games; music creation apps; content consumption apps; polling apps; map apps; etc.

[0105] The collaborative environment shown in FIG. **8** and described relative to FIG. **9A** can utilize breadcrumbs in one or more embodiments, and each of the breadcrumbs can be identified by a session identifier. A breadcrumb represents a converted message bubble and is identified by a session identifier that is shared with other message bubbles in the session. Each prior message bubble that is identified with the same session identifier as a new message bubble can be converted in one embodiment into a breadcrumb which can appear differently than the original content shown in the message transcript. As each new message bubble in the session arrives or is added to the message transcript, the prior message bubble identified with the same session identifier can be converted into a breadcrumb in one embodiment, and this is shown in the message transcripts

shown in FIGS. 9D, 9E, 9F, 9G, and 9H. FIGS. 9D and 9H show a message transcript **510** on Joe's device, and FIGS. 9E, 9F, and 9G show a message transcript **17** on Lester's device. Message transcript **510** shows Lester's user name **511**, and message transcript **517** shows Joe's user name **518**. In the example shown in FIGS. 9D, 9E, 9F, 9G, and 9H, Lester and Joe are engaged in a text messaging conversation and are each using an extension app, such as an image creation app or other extension apps. For example, Lester can be using extension app **407** shown in FIG. 8 and Joe can be using extension app **413** shown in FIG. 8. Lester's device can be using messaging app **409** while Joe's device can be using messaging app **411**. Referring back to FIG. 9D, it can be seen that the message transcript **510** includes message bubble **512** and message bubble **513** which indicate the content of the conversation within the message transcript **510**. In addition, Joe has used the extension app **413** to create content **515** which is appended to message bubble **514**. For example, Joe may have entered text as a text message and also created content using the extension app **413** and then cause both the text shown within message bubble **514** and the content **515** to be sent to Lester's device. The right side of the message transcript **510** shows messages that were sent by Joe while the left side of the message transcript **510** shows messages that were received from Lester. Referring now to FIG. 9E, it can be seen that message bubble **513** is now on the right side of the message transcript **517** while message bubble **514** and content **515** are on the left side of the message transcript **517** on Lester's device. Hence, Lester's device has received the text message within message bubble **514** and has also received the content generated by the extension app on Joe's device. Lester can then tap on the content **515** to cause the launching of the corresponding or counterpart extension app on Lester's device. The content **515** is associated with a session identifier which is maintained for the extension app on Lester's device. For example, at this point in the process, operation **461** shown in FIG. 9A can be performed on Lester's device as a result of the user selecting the content **515** which can be displayed within a message bubble in one embodiment. Lester can then use the extension app on Lester's device to create modified content or new content and send that modified or new content back to Joe. In FIG. 9F, it can be seen that Lester has created modified or new content **521** which is shown within a staging area **519** which shows text and other content that is ready to be sent and will be sent in response to selection by a user of a send command, such as the send button **523** shown in FIG. 9F. When Lester selects the send command with the content shown in the staging area **519**, this causes the text message **520** and the new or modified content **521** to be sent to Joe, and this can be seen in FIG. 9G in which message bubble **521A** shows the text message **520** on the right side of the message transcript **517** which also shows the content **521** which was modified or created as new by Lester using the extension app **407** on Lester's device.

[0106] It can be seen from FIG. 9G that content **515** has now been converted into a breadcrumb **515A**. This conversion in one embodiment can be performed by the messaging app or alternatively by the extension app. In one embodiment, the extension app can provide the text which appears within the breadcrumb **515A**, and the messaging app will use the session identifier to identify the message bubble that will be converted into a breadcrumb and this will cause the conversion of content **515** into the breadcrumb **515A** and display the breadcrumb next to the associated message bubble **514** without converting that message bubble **514** into a breadcrumb in one embodiment. Thus, FIG. 9G shows how a session identifier associated with prior message bubbles or content created by an extension app can be used to convert the prior message bubble(s) into a breadcrumb on a sending device. FIG. 9H shows how the conversion is displayed in one embodiment on the receiving device. In the figure shown in FIG. 9H, the message bubble **520A** is shown on the left side of the message transcript **510** along with the new or modified content from extension app **407** from Lester's device. The content **515** on Joe's device has been converted into a breadcrumb **515A** on the right side of the message transcript and is displayed adjacent to the message bubble **514** which accompanied the content **515** when the content **515** was originally sent.

[0107] If the receiving device, such as client device **405** in operation **459** is capable of installing

and using the extension app (identified by the app identifier provided in communication **419**) but that the extension app is not installed on the receiving device, the receiving device can, within the user interface of the messaging app offer to download and install the extension app (again specified by the app identifier in communication **419**, on the receiving device). FIG. **9C** shows an example in which the content from extension **407** can be displayed on the client device **405** in a message bubble **471** and the client device **405** is in the process of downloading and installing the extension app identified by the app identifier in communication **419**. In the example shown in FIG. **9C**, the content is displayed within the message bubble **471** while the extension app **413** is being installed. In one embodiment, a progress bar **473** (or alternatively a progress circle) can be shown within the message bubble **471** during the installation process. In one embodiment, the downloading and installation process can be performed in the background while the messaging app remains the foreground application. If the receiving device is not capable of installing or using the extension app, then the resource locator and the metadata can be passed in one embodiment to a web browser on the receiving device, and the web browser can become the foreground app and allow for the user interaction with the web page referred to by the resource locator.

[0108] In some embodiments, it may be desirable to provide an identifier of each user to each extension app executing on a client device, particularly in the case of a collaborative environment in which two or more users are interacting through the messaging apps and the extension apps. FIG. **10** shows an example of a method to provide an identifier for each extension app without causing a loss of the user's privacy. The method shown in FIG. **10** can be performed by each messaging app on each client device. In one embodiment, the method can be performed in response to a call from an extension app to an application programming interface (API) which can allow the extension app to request an identifier of the local user. In one embodiment, the identifier can be the local user's email address or phone number or other identifier used by the messaging system. The identifier provided to the extension app is an obfuscated identifier created, in one embodiment, by the method shown in FIG. **10**. In operation **501**, the messaging app can generate, in response to a call from the extension app, a salt for the particular extension app. In one embodiment, the salt can be a random number which is associated with that particular extension app. Then in operation **503**, the messaging app can generate a hash (such as a SHA-1 hash) of a combination of a user or device identifier and the salt. For example, the user identifier may be the phone number or email address of the user and that phone number or email address is combined with the salt and then the hash is created for that combination in operation **503**. Then in operation **505**, the hash is provided, through the IPC, to the extension app, and that hash value can be used as data with the resource locator which can then be sent to the other extension app to identify the user that has made changes or has created a content. In another embodiment, the messaging app can obfuscate the identifier by maintaining a mapping of the identifier and a randomly generated unique identifier for each extension app. In other words, for a given extension app, the messaging app can generate a random (and unique) identifier for the extension app and associate (e.g., map) that random identifier with the identifier of the user (e.g. local user's email address or phone number or other identifier used by the messaging system). This random identifier for the given extension app can be provided to the extension app, but the identifier is not provided to the extension app. Another extension app would receive a different, randomly generated identifier. This extension app specific identifier can then be provided to a counterpart extension app on another device so that the two extension apps can keep track of which users performed which actions in the context of a session or other collaborative environment.

[0109] In one embodiment, the messaging systems described herein can provide confirmation of receipt of messages to one or more extension apps, and this may be useful in certain cases in which the extension app needs to be assured that the remote extension app has the same known state as the local extension app. FIG. **11** shows an example of a method for providing confirmation of receipt. In operation **551**, the messaging app receives the "send" command from a user and in

response the local extension app provides the content to the messaging app. In turn, in operation 553, the messaging app sends the message and message bubble identifier and content and resource locator (if any) through the one or more messaging servers to a receiving device which also includes a message app. At some point, the messaging app on the local device in operation 555 receives a confirmation of receipt of the message and content and optional resource locator, and then can pass, in operation 557, through the IPC the confirmation of receipt to the local extension app so that the local extension app knows that the remote extension app has the same known state when the extension app provided its content for transmission to the remote extension app.

[0110] FIG. 12 shows another aspect to the embodiments described herein, and that aspect relates to the presentation of multiple message bubbles within a message transcript where different message bubbles have content created by different extension apps, and at least one of which may be executing (and have its content displayed in a compact view) while the message transcript of the user interface of the messaging app is also displayed. FIG. 12 shows an example of such an aspect. The communication device 600 in this embodiment includes a touch screen 601 which displays a message transcript 603 which includes two message bubbles 605 and 607. In addition, the messaging app presents a compact view 609 of an extension app, which in this case is an extension app for a restaurant reservation. In the example shown in FIG. 12, the extension app for a restaurant reservation has been used to send a message showing the content of a restaurant reservation, which is depicted as message bubble 607. This may have occurred as a result of receiving a message from another user who uses another communication device which provides the content of a restaurant review from another extension app. In the example shown in FIG. 12, message bubble 605 shows the content created by a remote extension app for providing restaurant reviews which is different than the extension app for making a restaurant reservation. In one embodiment, both extension apps may be executing on both communication devices that are part of the messaging session or chat.

[0111] Another aspect of the embodiments described herein relate to backward compatibility, and that aspect is shown in FIGS. 13A and 13B. Certain older devices may not be compatible with the extension app architecture described herein or may not use this architecture or may not even be smartphones. In one embodiment, a client device which is sending the message may be able to provide alternative content, either automatically or based upon a determination that the receiving device is not compatible with extension app. A method for providing backward compatibility is shown in FIG. 13A. In operation 651, an extension app on a sending device creates content on the sending device (first device) for delivery to a second device through a messaging app and a messaging service. This can be similar to operation 451 in FIG. 9A. The content created in operation 651 can then be passed through an interprocess communication to the messaging app in operation 653 in response to the messaging app receiving the user's "send" selection. Then in operation 655, the messaging app can display the content on the first device and also determine that the second device is not compatible with extension apps. In one embodiment, this can be determined as a result of receiving information about the receiving device (second device) from one or more messaging servers, such as the one or more messaging servers 51 shown in FIG. 2 which can maintain information about the state of each device, such as the version of the operating system of the device or the type of device, etc. As a result of determining that the second device is not compatible with extension apps, the messaging app can in operation 657 send alternative content to the second device, and FIG. 13B provides an example of this alternative content.

[0112] The communication device 675 shown in FIG. 13B may be, for example, an older smartphone which uses an older operating system that is not compatible with extension apps described herein. Nevertheless, the communication device 675 includes a touch screen 679 and a text entry region 683 and an on-screen keyboard 685 that provide functionality of a messaging app which also displays a message transcript 681 which includes a message bubble 687. The message bubble 687 contains the alternative content provided by a sending message app on a remote device. In this case the content includes an image 689 and a resource locator 691 which may be user

selectable to cause the display of a web page which the resource locator **691** refers to. In other words, the resource locator **691** may be user selectable to invoke a web browser on the communication device **675** to allow the user of the communication device **675** to interact through a web browser with the web page which may have the same affect in certain instances as an extension app interacting with the website.

[0113] Another aspect of the embodiments described herein relate to a service, such as an app marketplace that can provide a plurality of different extension apps for use within a messaging app according to the one or more embodiments described herein. The service or app marketplace can present browsable views of the plurality of different extension apps and messaging app plug-ins and provide information about those various extension apps and provide for downloading of those extension apps to a client device to allow the client device to install one or more extension apps. FIG. **14A** shows an example of a method for using such a service or app marketplace and FIG. **14B** shows an example of a user interface for a messaging extension app marketplace. The app marketplace shown in FIG. **14B** may be invoked in one embodiment from a browsable view of installed extension apps within the user interface of the messaging app on a client device. For example, the selection of icon **167** shown in FIG. **4B** may cause the presentation of the messaging extension app marketplace shown in FIG. **14B**. The user can then browse through the one or more collections of messaging extension apps and select one or more extension apps which may be free or may be purchased. In the example shown in FIG. **14B**, the messaging extension app marketplace **725** can include a navigation bar **729** and a browsable view of one or more messaging extension apps, such as apps **726**, **728**, and **731** displayed on the touch screen **727**. The user can browse through the apps by swiping, in one embodiment, the user's finger across the touch screen or by the use of a navigation bar **729**. The user can then select the download and install one or more of the extension apps, and as a result the messaging app on the user's client device can add an icon representing the newly installed extension app into the browsable view of installed extension apps, such as the browsable view **157** shown in FIG. **4B**. In addition, the messaging app can add the installed extension app into a list of installed extension apps along with the identifier of the app ("app identifier") provided by the app marketplace. While the user's use of the app marketplace shown in FIG. **14B** is one way of installing extension apps, another way is shown in FIG. **14A** in which the installation process begins as a result of the user interacting with a message in the message transcript of a messaging app.

[0114] Referring now to FIG. **14A**, a messaging app in operation **701** can receive content (and an optional resource locator and data) created by an extension app on a remote device, such as a first device, and can also receive an app identifier of the extension app on the remote device. In one embodiment, the app identifier can be the same identifier provided by the app marketplace when the extension app was installed on the first device or can be a different identifier that is associated with the app store's identifier. Then in operation **703**, the messaging app on the second device can display the content in a message bubble and can retain the resource locator and data if provided. At this point, the messaging app in this embodiment does not attempt to launch an extension app which in fact is not installed on the second device at this point in time. Then in operation **705**, the messaging app receives a selection, such as a tap, of the message bubble containing the content provided by the extension app from the first device, and the messaging app determines that the extension app (as identified by the received app identifier) is not installed on the second device. At this point, as shown in operation **707** of FIG. **14A**, the messaging app offers to download and install the extension app on the second device and the user can select to install that extension app which was identified by the app identifier received in operation **701**. In some instances, the user may need to purchase the extension app before it can be downloaded and installed. The offer to download and install may be presented in a notice, such as the notice **259** shown in FIG. **5F**, and this notice may include a selectable option to cause the downloading and the installing of the extension app for the selected message bubble. Then in operation **709**, the second device

downloads and installs the extension app and can launch the newly installed extension app. In one embodiment, the download and install can occur in the background while the messaging app remains the foreground. In one embodiment, the progress of the download and install can be shown in a progress bar, such as the progress bar **473** shown within the selected message bubble. After operation **709** is completed, then the newly installed extension app can be used on the second device and, the messaging app can provide, in operation **711**, the content and optional resource locator and data to the extension app on the second device within the view hosted by messaging app.

[0115] In one embodiment, the messaging app may cause the automatic updating of extension apps which have been installed. In another embodiment, the messaging app may provide alerts or notices to the user that certain extension apps are in need of being updated and the notifications about these updates can be received from an extension app marketplace in one embodiment. This can allow a user to selectively decide whether or not to update specific messaging apps.

[0116] Another aspect of the embodiments described herein relates to the conversion of a resource locator, such as a URL, that was entered as text by, for example, a user (or another application) into a messaging app, and FIG. **15A**, **15B**, **16**, **17**, and **18** shows examples of this aspect. Referring to FIG. **15A**, the communication device **800** includes a messaging app that presents the user interface shown in FIG. **15A**, and this user interface is displayed on the touch screen **801** and includes an on-screen keyboard **802**, a text staging/entry region **803**, and a messaging transcript **805**. The message transcript **805** includes three message bubbles, two from the user of the communication device **800** on the right side of the message transcript **805** and one from the other user which is in a chat or conversation with the user of the communication device **800**. In particular, message bubbles **806** and **807** originated from the user of the communication device **800** while message bubble **804** originated with the other user who is in communication with the user who is using the communication device **800**. The first, in time, message shown in the message transcript **805** is the message bubble **804**, and the second message, in time, is shown by the message bubble **806** and is entered by the user of the communication device **800**. Next, in time, the user of the communication device **800** either types out or speaks or otherwise causes the entry of the URL (or other resource locator) shown in text bubble **807**. For example, the user could type out the URL or speak the URL or other resource locator or copy and paste the URL from a web browser or other source of the URL (or other resource locator). In the example shown in FIG. **15A**, the message contained within message bubble **807** has not yet been sent, but the user has caused the entry of the text which constitutes the URL (or other resource locator) as shown in FIG. **15A**. In one embodiment, when the user of communication device **800** selects the send command then the URL or other resource locator within the text bubble **807** can be converted to an image or other content such as the image shown in bubble **807a** in FIG. **15B**. In particular, the user of communication device **800** can, after entry of the URL or other resource locator within text staging/entry region **803** select the “send” command by, for example, tapping on the send icon or send button or by other techniques known in the art to indicate to the communication device **800** that the user has decided to send the message. In one embodiment, in response to the selection of the send command, the messaging app which controls the user interface shown in FIG. **15A** can then perform the method shown in FIG. **16** to convert the URL (or other resource locator) into an image or other content, such as the image shown in message bubble **807A** in the message transcript **805**.

[0117] Referring now to FIG. **16**, a messaging app on the sending device (such as the communication device **800**) can receive text from a user for delivery to a second device that includes a second messaging app. This is shown as operation **851** in which the first messaging app on the first device receives the text from the user, which may include one or more URLs (or other resource locators). Then in operation **853**, the first messaging app can attempt to detect one or more URLs (or other resource locators) in the received text. In one embodiment, the first messaging app may use one or more conventional data detectors which are configured to detect URLs (or other

resource locators) in the received text. The data detectors can be, for example, the data detectors **892** which are a module of one or more text processors that are designed to recognize text patterns that conventional URLs (or other resource locators) employ. Operation **855** can follow operation **853** and in one embodiment can be an optional operation, although it may be desirable to perform this operation in order to filter out conversions which may not be appropriate. In operation **855**, the messaging app, such as the first messaging app, determines whether to convert the detected URL (or other resource locators) to an image or other content. In one embodiment, a set of rules or heuristics to determine whether or not to do the conversion can be used. FIG. **17** shows an example of two rules **875** and **879** that can be used in one embodiment. Rule **875** informs the first messaging app of when to convert if the text contains only one URL (or other resource locators) at either the beginning of the text or the end of the text. In another embodiment, rule **875** can be modified to also require that the URL (or other resource locators) start with the characters “http://”. Rule **879** instructs the first messaging app to convert a detected URL if the text that is entered by the user contains only one or more URLs (or other resource locators) and white spaces. In one embodiment, both of these rules can be used in the optional operation **855**.

[0118] Referring back to FIG. **16**, operation **857** can follow operation **855** if operation **855** is used or can follow operation **853**. In operation **857**, the first messaging app retrieves at least a portion of a web page or other resource referred to by the URL (or other resource locators). For example, the first messaging app can retrieve the web page shown by the text or URL (or other resource locators) within message bubble **807**. Then in operation **859** a web page processing module, such as web page processing module **894** can then process the retrieved contents of the web page to identify a representative image for the web page. In one embodiment, the representative image can be specified by metadata or metatags in the web page itself. If no such representative image is specified, in one embodiment the first messaging app can use other icons such as the favicon of the page as a default representative image. Once the first messaging app has identified a representative image for the web page, it can then request the representative image (e.g. using the URL for the image) and download the requested representative image. Then in operation **861**, the first messaging app can send the representative image to the second messaging app through one or more messaging servers. If the user entered other text, such as the text “check this out” shown in message bubble **806**, then that text is also sent to the second messaging app through the one or more messaging servers. The result of operation **861** causes the representative image and any associated non-URL text to be sent and also causes the presentation of the user interface shown in FIG. **15B** in which the message transcript **805** now includes message bubble **807A** which replaces message bubble **807** in the message transcript **805**. This shows the sender the content of the message bubble which will also appear on the receiver's device when the second messaging app on the second device receives that message bubble. In one embodiment, the message bubble corresponding to the message bubble **807A** will appear on the left side of the message transcript displayed by the second messaging app on the second device. Operation **861** can be implemented by delaying the sending of the non-URL text for a predetermined period of time (such as 5 seconds) after the send command has been received while attempting to retrieve the representative image; if the representative image is retrieved within the predetermined period of time, then the non-URL text and the representative image are sent (and can be sent by the sending device in one message). The predetermined period of time gives the sending device time to retrieve the representative image. If the representative image is not retrieved within the predetermined period of time, then the non-URL text is sent to the second messaging app on the second device while the sending device continues attempting to retrieve the representative image. The second message app can receive the non-URL text and a message that instructs the second messaging app to display an indicator (e.g. a spinning ball, etc.) that the second messaging app is awaiting additional content for the message. If after another predetermined period of time (such as 30 seconds) the representative image has still not been retrieved then the first messaging app aborts the process of trying to send the

representative image and the second messaging app removes the indicator.

[0119] In one embodiment, the first messaging app in operation **861** can optionally encrypt the image and the other data such as the non-URL text before sending the image and the other data to the second messaging app through the one or more messaging servers. This encryption can be performed for example by encryption module **896** shown in FIG. **18**.

[0120] Referring now to FIG. **18**, the text messaging module **891** in the communication device **890** is an embodiment of a module which can display the user interfaces shown in FIG. **15A** and FIG. **15B**, and that text messaging module **891** can interact with several other modules including the data detectors module **892**, the web page processing module **894**, the heuristics module **895**, and the encryption module **896**. In addition, the text messaging module **891** can send encrypted messages as described herein through the network interface **893** which can in one embodiment be a set of one or more radios, such as cellular telephone wireless radios or WiFi radios, etc. The text messaging module **891** can use the encryption module **896** to perform the encryption described herein. Further, the text messaging module **891** can use the heuristics module **895** which can be configured to use the heuristics shown in FIG. **17** in one embodiment. The web processing module **894** can be in one embodiment conventional software for processing a web page such as WebKit.

[0121] Another aspect of the embodiments described herein relate to media extension apps (or plug-ins) and the conversion of URLs (or other resource locators) that refer to media content in a media catalog, and FIGS. **19A**, **19B**, **20A**, **20B**, **20C**, **21A**, **21B**, **21C**, **22**, and **23** show examples of these embodiments. FIG. **19A** shows an example of a user interface for a messaging app after a user has entered a URL (or other resource locator) in the messaging app, where the URL (or other resource locator) refers to one or more media content in a media catalog, such as the iTunes media catalog or the Apple Music media catalog. The communication device **950** can include a touch screen **901** which displays the user interface of a messaging app that is executing on the communication device **950**. The user interface can include an on-screen keyboard **902** and a text entry region **903** and a message transcript **905**. The message transcript **905** shows three message bubbles **904**, **906**, and **907**. In the example shown in FIG. **19A**, message bubble **904** originated from the other user in the chat or conversation while message bubbles **906** and **907** originated from the user of the communication device **950** in one embodiment. It can be seen that the user of communication device **950** has entered a URL (or other resource locator) into the message bubble **907**. This URL may be entered, for example, by the user typing the URL or by the user speaking the URL or by the user copying and pasting the URL into the text entry region **903** or by other techniques in the art. In one embodiment, the URL shown within message bubble **907** has not been sent by the user of communication device **950**. In one embodiment, when the user decides to send the URL by, for example, tapping on a send button or send icon, the messaging app executing on communication device **950** can perform the method shown in FIG. **20B** within the system shown in FIG. **20A** to convert the URL, which refers to media content in a media catalog, into metadata related to that media content such as the artwork **911** and the album title **913** shown in FIG. **19B**. After the conversion of the URL which refers to media content in a media catalog, the message bubble on the sender's device transforms from message bubble **907** in the message transcript **905** to the message bubble **907A** shown in FIG. **19B** which appears within the message transcript **905** in one embodiment.

[0122] Referring now to FIG. **20B**, operation **980** is performed by the first messaging app on the first device which can be the communication device **950** which receives text from a user for delivery to a second device which includes a second messaging app. In one embodiment, the text can include a URL (or other resource locator) that refers to media content in a media catalog, such as the URL within the message bubble **907** in FIG. **19A**. Then in operation **982**, the text entered in operation **980** can be examined by one or more data detectors which are configured to detect a URL (or other resource locator) referring to media content such as audio content (e.g. a song or album of songs) or audio visual content (e.g. a movie or television show, etc.) in a catalog of media content.

In one embodiment, the data detectors can be the data detectors **1005** shown in FIG. **20C**. In one embodiment, these media content data detectors can be data detectors that are known in the art to detect URLs (or other resource locators) and in particular to detect the particular known pattern of characters that appear in known URLs (or other resource locators) from media catalogs, such as the media catalog for iTunes or a media catalog for Apple Music or other known media catalogs (such as a Spotify catalog, a Google Play catalog, etc.). In one embodiment, operation **982** may optionally use heuristics to decide whether or not to convert the detected URLs (or other resource locators). Examples of these heuristics are provided herein and may be used in certain cases in one embodiment as part of or after operation **982**.

[0123] Then in operation **986**, the first messaging app on the first device can transmit, in response to detecting the URL, at least a portion of the URL to one or more servers that are coupled to a source of data about the catalog. This is shown in FIG. **20A** in which the communication device **950** transmits at least a portion of the URL to a media store and/or store services set of servers **962** which are configured to respond to the URLs from a messaging app, such as the first messaging app, with metadata about the media content referred to by the URL. In one embodiment, a media store can include a set of one or more store services servers that are configured to perform lookup operations based on at least portions of the URLs transmitted from messaging apps, such as the first messaging app on communication device **950**. These store services servers **962** can perform lookup operations using one or both of the catalog databases **963** and **964**. In one embodiment, the catalog database **963** can be a database for the catalog of content on the iTunes store, which is media content that can be purchased and stored locally (after downloading the content) on one or more devices that are authorized under the account. The catalog database **964** in one embodiment can be a catalog that “rents” media content on the basis of a subscription, such as the Apple Music catalog or the Spotify catalog, etc. In one embodiment, the store services servers **962** can perform lookup operations on these catalog databases, where the input to the lookup operation is a portion or the entire URL transmitted from the messaging apps, and these store services servers **962** can then return the results retrieved from those lookup operations to the first messaging app. In one embodiment, those results can be a dictionary of metadata about the media content. Operation **988** in FIG. **20B** occurs when the first messaging app receives metadata, such as the dictionary of metadata about the media content from these store services servers. Operation **988** can also include processing by the first messaging app to extract appropriate metadata from the dictionary of metadata and then using the extracted metadata to retrieve other metadata, such as artwork, etc. about the media content from an external source such as the store services servers **962**. In an alternative embodiment, the metadata received initially in operation **988** may contain all of the content needed by the first messaging app without requiring further processing of the metadata dictionary or further downloads from the store services server. In one embodiment, the results of the operation **988** provides the first messaging app with a collection of metadata about the media content, such as the artwork for the content, the album title, the artist name, genre category, etc. Then in operation **990**, the first messaging app can transmit this metadata about the media content to a second messaging app through one or more messaging servers, such as the messaging servers **965** shown in FIG. **20A**. The second messaging app on the receiving device, such as the second device **967** can then present a message bubble which is similar to the message bubble shown in FIG. **19B**, and this message bubble can include the artwork **911** and the album title **913** as well as a play button or a preview button **912** (rather than containing the actual text entered by the user of communication device, which is shown within message bubble **907** in FIG. **19A**). While the example shown in FIG. **19B** shows a music album, it will be appreciated that other types of content can also be processed in the method shown in FIG. **20B**, and the metadata about these other types of content can be displayed within a message bubble such as message bubble **907a**. These other types of content can include: radio stations, TV shows, movies, playlists, books, etc.

[0124] FIG. **20C** shows an example of a communication device **1001** which can represent the

communication device **950** and which includes several modules that can be configured to perform the method shown in FIG. **20B**. In particular, text messaging module **1003** can be a messaging app, such as the first messaging app. The text messaging module can present the user interface shown in FIG. **19A** and use data detectors, such as media content data detectors **1005** as part of operation **918** in FIG. **20B**. The text messaging module **1003** can also operate with a dictionary to metadata converter module **1009** which can perform the processing of the dictionary of metadata received in one embodiment from the store services servers **962**, which dictionary is processed to extract appropriate metadata and which can be used to retrieve further metadata from a set of servers, such as the store services server **962**. The text messaging module **1003** can also operate with an encryption module **1011** to encrypt the metadata which is transmitted in operation **990** to the second messaging app through a set of one or more messaging servers, such as the messaging servers **965**. In one embodiment, the text messaging module and the converter module **1009** and the data detector module **1005** can be implemented at least in part with software. Similarly, the encryption module **1011** can also be implemented at least in part with software although it may include hardware acceleration such as dedicated hardware encryption components. The communication device **1001** also includes one or more radios or other network interfaces **1007** to allow the communication device **1001** to transmit and receive.

[0125] Another aspect of the embodiments described herein relate to media extension apps, such as music extension apps, or music plug-ins for a messaging app which can provide playback functionality, sharing functionality, search functionality, and other functionality related to media content. It will be appreciated that the term music extension app refers to an extension app that is configured to operate with media content which can be more than just music; in other words, it will be appreciated that the term music extension app also includes an extension app that can provide content other than music or songs, such as movies, television shows, podcasts, radio broadcasts, etc. Similarly, a music plug-in also refers to a plug-in that is configured to operate with these various different types of content other than just music or songs.

[0126] FIG. **21A** shows an example of a user interface displayed by a messaging app, such as a first messaging app configured to transmit and receive messages to other messaging apps. The messaging app executing on communication device **1025** displays a user interface that includes a message transcript **1031**, a user name **1050A**, two message bubbles **1037** and **1039** within the message transcript **1031**, a text entry region **1029**, and the messaging application displays on touch screen **1027** a view of a media extension app **1033** which replaces the on-screen keyboard of the messaging app in the example shown in FIG. **21A**. The media extension app **1033** can be similar to the extension apps described herein and can communicate with the messaging app through IPC in the same manner that is described above. For example, the media extension app **1033** can be similar to the extension app **83** and can communicate with the messaging app through the IPC framework or other functionality to provide content or other data to the messaging app as well as receive data from the messaging app, such as messaging app **76** shown in FIG. **3A**. The messaging app executing on communication device **1025** can also be similar to the messaging app **76** described above. The messaging app on communication device **1025** hosts a view of the media extension app **1033** which presents a currently playing item or a list of recently played media items, such as recently played item **1041** in the area that has replaced the on-screen keyboard of the messaging app. In one embodiment, the view presented by the media extension app **1033** can be a browsable view in which a user can scroll back and forth or up and down in order to see recently played media items which can be media items within the media catalog or set of media catalogs, such as the iTunes catalog and/or the Apple Music catalog. In addition, the media extension app **1033** can also show in a view hosted by the messaging app, an icon of the currently playing media item. In one embodiment, the user of communication device **1025** can share selected media items to a user in a conversation through the messaging app by selecting one or more of recently played items (or a currently playing item) within the view provided by the media extension app **1033** at the

bottom of the touch screen **1027**. For example, the user of communication device can share with the user having the user name **1050A** the recently played item **1041** shown at the bottom of FIG. **10A** by selecting that recently played item (or if the view shows the currently item playing item, then the currently playing item can be shared). In one embodiment, the user can tap or otherwise select the recently played item **1041** (or a currently playing item), and in turn the media extension app communicates metadata through the IPC to the messaging app to cause the presentation of the message bubble **1045** shown in FIG. **21B** in the message transcript **1031**. In another embodiment, a user can select an item, such as a recently played item and then select a “share” button to cause the item to be shared through the messaging app. In other words, the media extension app has used a communication process through the IPC with the first messaging app executing on communication device **1025** to provide the metadata for the message **1045**. In one embodiment, this metadata can include the artwork **1042** and the artist/album title **1043** which is displayed within the message bubble **1045**. In addition, the message bubble **1045** can include a play button **1044** which also appears on the receiving device as shown in FIG. **21C** and the selection of the play button **1044** can cause the device to play the media item associated with the message bubble **1045**. This metadata can be locally stored and provided to the first messaging app or the method shown in FIG. **20B** can be used to obtain the metadata . . .

[0127] FIG. **21C** shows an example of a second messaging app which has received the metadata from a first messaging app such as the messaging app executing on communication device **1025**. The communication device **1050** shown in FIG. **21C** includes a messaging app that displays the user interface shown on the touch screen **1054**. This user interface can include an on-screen keyboard **1053** and a text entry region **1052** and a message transcript **1051** which displays two message bubbles **1039A** and **1045A**. The second messaging app which displays the user interface shown in FIG. **21C** on communication device **1050** can perform the method shown in FIG. **22**. In one embodiment, this method can be performed even if a counterpart media extension application which has the same app identifier as the media extension app **1033** is not installed on the communication device **1050**. The second messaging app which can execute on communication device **1050** can receive in operation **1101** the metadata about the media content in the catalog which was sent by the first messaging app on communication device **1025**. This results in the display of the message bubble **1045** which includes the artwork **1042** and the artist and album title **1043** along with a play or preview button **1044**. In one embodiment, a selection of the play (or preview) button **1044** can cause the playback of the entire content (or in the case of the preview button, playback of a preview (a portion) of the content. In operation **1103**, the second messaging app can determine the subscription or account status with each of the appropriate catalogs to determine which options or buttons to display in the user interface. In particular, in operation **1105** a preview option or button can be presented if the user of the communication device **1050** does not have subscription. Alternatively, if the user does have a subscription then the messaging app can present a play option or button in operation **1105**. Table **1110** shown in FIG. **23** shows an example of a set of rules which determine which options are displayed for different types of content and for different statuses of the account. In particular, the rows **1111** and **1112** and **1113** represent different account statuses and columns **1115**, **1116**, **1117**, and **1118** indicate the status of the media content relative to two different media catalogs. For example, if the user of communication device **1050** is a signed-in subscriber (row **1111**) then in two of the four cases a play button is displayed (columns **1115** and **1116**). If the media content is only in the purchase catalog then the display will show the play button only if the content has already been purchased and if it has not then a preview button or option is displayed. If the media content is not in either catalog then only the metadata is displayed if it is available.

[0128] In one embodiment, the second messaging app executing on communication device **1050** can present at least a preview of the media content even if a media extension app is not installed on the communication device **1050**. In one embodiment, the second messaging app on communication

device **1050** can also display a subscription option which allows a user to start a subscription to the content.

[0129] One or more Application Programming Interfaces (APIs) may be used in some embodiments. An API is an interface implemented by a program code component or hardware component (hereinafter “API-implementing component”) that allows a different program code component or hardware component (hereinafter “API-calling component”) to access and use one or more functions, methods, procedures, data structures, classes, and/or other services provided by the API-implementing component. An API can define one or more parameters that are passed between the API-calling component and the API-implementing component.

[0130] An API allows a developer of an API-calling component (which may be a third party developer) to leverage specified features provided by an API-implementing component. There may be one API-calling component or there may be more than one such component. An API can be a source code interface that a computer system or program library provides in order to support requests for services from an application. An operating system (OS) can have multiple APIs to allow applications running on the OS to call one or more of those APIs, and a service (such as a program library) can have multiple APIs to allow an application that uses the service to call one or more of those APIs. An API can be specified in terms of a programming language that can be interpreted or compiled when an application is built.

[0131] In some embodiments the API-implementing component may provide more than one API, each providing a different view of or with different aspects that access different aspects of the functionality implemented by the API-implementing component. For example, one API of an API-implementing component can provide a first set of functions and can be exposed to third party developers, and another API of the API-implementing component can be hidden (not exposed) and provide a subset of the first set of functions and also provide another set of functions, such as testing or debugging functions which are not in the first set of functions. In other embodiments the API-implementing component may itself call one or more other components via an underlying API and thus be both an API-calling component and an API-implementing component.

[0132] An API defines the language and parameters that API-calling components use when accessing and using specified features of the API-implementing component. For example, an API-calling component accesses the specified features of the API-implementing component through one or more API calls or invocations (embodied for example by function or method calls) exposed by the API and passes data and control information using parameters via the API calls or invocations. The API-implementing component may return a value through the API in response to an API call from an API-calling component. While the API defines the syntax and result of an API call (e.g., how to invoke the API call and what the API call does), the API may not reveal how the API call accomplishes the function specified by the API call. Various API calls are transferred via the one or more application programming interfaces between the calling (API-calling component) and an API-implementing component. Transferring the API calls may include issuing, initiating, invoking, calling, receiving, returning, or responding to the function calls or messages; in other words, transferring can describe actions by either of the API-calling component or the API-implementing component. The function calls or other invocations of the API may send or receive one or more parameters through a parameter list or other structure. A parameter can be a constant, key, data structure, object, object class, variable, data type, pointer, array, list or a pointer to a function or method or another way to reference a data or other item to be passed via the API.

[0133] Furthermore, data types or classes may be provided by the API and implemented by the API-implementing component. Thus, the API-calling component may declare variables, use pointers to, use or instantiate constant values of such types or classes by using definitions provided in the API.

[0134] Generally, an API can be used to access a service or data provided by the API-implementing component or to initiate performance of an operation or computation provided by the API-

implementing component. By way of example, the API-implementing component and the API-calling component may each be any one of an operating system, a library, a device driver, an API, an application program, or other module (it should be understood that the API-implementing component and the API-calling component may be the same or different type of module from each other). API-implementing components may in some cases be embodied at least in part in firmware, microcode, or other hardware logic. In some embodiments, an API may allow a client program (e.g., game center application) to use the services provided by a Software Development Kit (SDK) library. In other embodiments an application or other client program may use an API provided by an Application Framework. In these embodiments the application or client program may incorporate calls to functions or methods provided by the SDK and provided by the API or use data types or objects defined in the SDK and provided by the API. An Application Framework may in these embodiments provide a main event loop for a program that responds to various events defined by the Framework. The API allows the application to specify the events and the responses to the events using the Application Framework. In some implementations, an API call can report to an application the capabilities or state of a hardware device, including those related to aspects such as input capabilities and state, output capabilities and state, processing capability, power state, storage capacity and state, communications capability, etc., and the API may be implemented in part by firmware, microcode, or other low level logic that executes in part on the hardware component.

[0135] The API-calling component may be a local component (i.e., on the same data processing system as the API-implementing component) or a remote component (i.e., on a different data processing system from the API-implementing component) that communicates with the API-implementing component through the API over a network. It should be understood that an API-implementing component may also act as an API-calling component (i.e., it may make API calls to an API exposed by a different API-implementing component) and an API-calling component may also act as an API-implementing component by implementing an API that is exposed to a different API-calling component.

[0136] The API may allow multiple API-calling components written in different programming languages to communicate with the API-implementing component (thus the API may include features for translating calls and returns between the API-implementing component and the API-calling component); however the API may be implemented in terms of a specific programming language. An API-calling component can, in one embodiment, call APIs from different providers such as a set of APIs from an OS provider and another set of APIs from a plug-in provider and another set of APIs from another provider (e.g. the provider of a software library) or creator of the another set of APIs.

[0137] FIG. 24 is a block diagram illustrating an exemplary API architecture, which may be used in one embodiment of the present invention. As shown in FIG. 24, the API architecture 3200 includes the API-implementing component 3210 (e.g., an operating system, a library, a device driver, an API, an application program, software or other module) that implements the API 3220. The API 3220 specifies one or more functions, methods, classes, objects, protocols, data structures, formats and/or other features of the API-implementing component that may be used by the API-calling component 3230. The API 3220 can specify at least one calling convention that specifies how a function in the API-implementing component receives parameters from the API-calling component and how the function returns a result to the API-calling component. The API-calling component 3230 (e.g., an operating system, a library, a device driver, an API, an application program, software or other module) makes API calls through the API 3220 to access and use the features of the API-implementing component 3210 that are specified by the API 3220. The API-implementing component 3210 may return a value through the API 3220 to the API-calling component 3230 in response to an API call.

[0138] It will be appreciated that the API-implementing component 3210 may include additional

functions, methods, classes, data structures, and/or other features that are not specified through the API **3220** and are not available to the API-calling component **3230**. It should be understood that the API-calling component **3230** may be on the same system as the API-implementing component **3210** or may be located remotely and accesses the API-implementing component **3210** using the API **3220** over a network. While FIG. **24** illustrates a single API-calling component **3230** interacting with the API **3220**, it should be understood that other API-calling components, which may be written in different languages (or the same language) than the API-calling component **3230**, may use the API **3220**.

[0139] The API-implementing component **3210**, the API **3220**, and the API-calling component **3230** may be stored in a machine-readable medium (e.g., computer-readable medium), which includes any mechanism for storing information in a form readable by a machine (e.g., a computer or other data processing system). For example, a machine-readable medium includes magnetic disks, optical disks, random access memory; read only memory, flash memory devices, etc.

[0140] In FIG. **25** ("Software Stack"), in one embodiment of the present invention, applications can make calls to Services A or B using several Service APIs and to Operating System (OS) using several OS APIs. Services A and B can make calls to OS using several OS APIs.

[0141] Note that the Service **2** has two APIs, one of which (Service **2** API **1**) receives calls from and returns values to Application **1** and the other (Service **2** API **2**) receives calls from and returns values to Application **2**. Service **1** (which can be, for example, a software library) makes calls to and receives returned values from OS API **1**, and Service **2** (which can be, for example, a software library) makes calls to and receives returned values from both OS API **1** and OS API **2**. Application **2** makes calls to and receives returned values from OS API **2**.

[0142] The systems and methods described herein can be implemented in a variety of different data processing systems and devices, including general-purpose computer systems, special purpose computer systems, or a hybrid of general purpose and special purpose computer systems. Exemplary data processing systems that can use any one of the methods described herein include desktop computers, laptop computers, tablet computers, smart phones, cellular telephones, personal digital assistants (PDAs), embedded electronic devices, or consumer electronic devices.

[0143] FIG. **26** is a block diagram of data processing system hardware according to an embodiment. Note that while FIG. **26** illustrates the various components of a data processing system that may be incorporated into a mobile or handheld device, it is not intended to represent any particular architecture or manner of interconnecting the components as such details are not germane to the present invention. It will also be appreciated that other types of data processing systems that have fewer components than shown or more components than shown in FIG. **26** can also be used with the present invention.

[0144] As shown in FIG. **26**, the data processing system includes one or more buses **1309** that serve to interconnect the various components of the system. One or more processors **1303** are coupled to the one or more buses **1309** as is known in the art. Memory **1305** may be DRAM or non-volatile RAM or may be flash memory or other types of memory or a combination of such memory devices. This memory is coupled to the one or more buses **1309** using techniques known in the art. The data processing system can also include non-volatile memory **1307**, which may be a hard disk drive or a flash memory or a magnetic optical drive or magnetic memory or an optical drive or other types of memory systems that maintain data even after power is removed from the system. The non-volatile memory **1307** and the memory **1305** are both coupled to the one or more buses **1309** using known interfaces and connection techniques. A display controller **1322** is coupled to the one or more buses **1309** in order to receive display data to be displayed on a display device **1323**. The display device **1323** can include an integrated touch input to provide a touch screen. The data processing system can also include one or more input/output (I/O) controllers **1315** which provide interfaces for one or more I/O devices, such as one or more mice, touch screens, touch pads, joysticks, and other input devices including those known in the art and output devices (e.g.

speakers). The input/output devices **1317** are coupled through one or more I/O controllers **1315** as is known in the art.

[0145] While FIG. **26** shows that the non-volatile memory **1307** and the memory **1305** are coupled to the one or more buses directly rather than through a network interface, it will be appreciated that the present invention can utilize non-volatile memory that is remote from the system, such as a network storage device which is coupled to the data processing system through a network interface such as a modem or Ethernet interface. The buses **1309** can be connected to each other through various bridges, controllers and/or adapters as is well known in the art. In one embodiment the I/O controller **1315** includes one or more of a USB (Universal Serial Bus) adapter for controlling USB peripherals, an IEEE 1394 controller for IEEE 1394 compliant peripherals, or a Thunderbolt controller for controlling Thunderbolt peripherals. In one embodiment, one or more network device(s) **1325** can be coupled to the bus(es) **1309**. The network device(s) **1325** can be wired network devices (e.g., Ethernet) or wireless network devices (e.g., WI-FI, Bluetooth).

[0146] It will be apparent from this description that aspects of the present invention may be embodied, at least in part, in software. That is, the techniques may be carried out in a data processing system in response to its processor executing a sequence of instructions contained in a storage medium, such as a non-transitory machine-readable storage medium (e.g. DRAM or flash memory). In various embodiments, hardwired circuitry may be used in combination with software instructions to implement the present invention. Thus the techniques are not limited to any specific combination of hardware circuitry and software, or to any particular source for the instructions executed by the data processing system. Moreover, it will be understood that where mobile or handheld devices are described, the description encompasses mobile devices (e.g., laptop devices, tablet devices), handheld devices (e.g., smartphones), as well as embedded systems suitable for use in wearable electronic devices.

[0147] The present disclosure recognizes that the use of personal information data, in the present technology, can be used to the benefit of users. For example, the personal information data can be used to deliver information or targeted content that is of greater interest to the user. Accordingly, use of such personal information data can enable calculated control of the delivered content. Further, other uses for personal information data that benefit the user are also contemplated by the present disclosure.

[0148] The present disclosure further contemplates that the entities responsible for the collection, analysis, disclosure, transfer, storage, or other use of such personal information data will comply with well-established privacy policies and/or privacy practices. In particular, such entities should implement and consistently use privacy policies and practices that are generally recognized as meeting or exceeding industry or governmental requirements for maintaining personal information data private and secure. For example, personal information from users should be collected for legitimate and reasonable uses of the entity and not shared or sold outside of those legitimate uses. Further, such collection should occur only after receiving the informed consent of the users. Additionally, such entities would take any needed steps for safeguarding and securing access to such personal information data and ensuring that others with access to the personal information data adhere to their privacy policies and procedures. Further, such entities can subject themselves to evaluation by third parties to certify their adherence to widely accepted privacy policies and practices.

[0149] Despite the foregoing, the present disclosure also contemplates embodiments in which users selectively block the use of, or access to, personal information data. That is, the present disclosure contemplates that hardware and/or software elements can be provided to prevent or block access to such personal information data. For example, in the case of health information or advertisement delivery services, the present technology can be configured to allow users to select to “opt in” or “opt out” of participation in the collection of personal information data during registration for services. In another example, users can select not to provide location information for targeted

content delivery services. In yet another example, users can select to not provide precise location information, but permit the transfer of location zone information.

[0150] In the foregoing specification, specific exemplary embodiments have been described. It will be evident that various modifications may be made to those embodiments without departing from the broader spirit and scope set forth in the following claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense.

Claims

1. A non-transitory machine readable medium storing executable program instructions which when executed by a data processing system cause the data processing system to perform a method for causing playback of a preview of media content in a messaging application, the method comprising: receiving, by the messaging application executing on a first device, a message from a second device, the message including metadata about the media content in a catalog; displaying, by the messaging application, the message in a user interface of the messaging application; and displaying, based on the metadata, a play button or a preview button within the message, the play button selectable to cause the playback of the media content and the preview button selectable to cause playback of a preview of the media content.
2. The non-transitory machine readable medium of claim 1, wherein the method further comprises: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; and responsive to the user of the first device having the subscription to the catalog, displaying the play button in the message.
3. The non-transitory machine readable medium of claim 1, wherein the method further comprises: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; determining, by the messaging application, whether a user of the first device is signed-in to the catalog; responsive to the user of the first device having the subscription to the catalog and being signed-in to the catalog, displaying the play button in the message; and responsive to the user of the first device having the subscription to the catalog but being signed-out to the catalog, displaying the preview button in the message.
4. The non-transitory machine readable medium of claim 1, wherein the method further comprises: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; and responsive to the user of the first device not having the subscription to the catalog, displaying the preview button in the message.
5. The non-transitory machine readable medium of claim 4, wherein the method further comprises displaying, in the message, a subscription option that is selectable to allow the user to start a subscription to the catalog.
6. The non-transitory machine readable medium of claim 1, wherein the play button or the preview button is displayed by a media extension application associated with the catalog executing on the first device.
7. The non-transitory machine readable medium of claim 1, wherein the play button or the preview button is displayed even if a media extension application associated with the catalog is not installed on the first device.
8. The non-transitory machine readable medium of claim 1, wherein the method further comprises, based on the metadata, displaying artwork associated with the media content in the message.
9. A method for causing playback of a preview of media content in a messaging application, the method comprising: receiving, by the messaging application executing on a first device, a message from a second device, the message including metadata about the media content in a catalog; displaying, by the messaging application, the message in a user interface of the messaging application; and displaying, based on the metadata, a play button or a preview button within the message, the play button selectable to cause the playback of the media content and the preview

button selectable to cause playback of a preview of the media content.

- 10.** The method of claim 9, further comprising: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; and responsive to the user of the first device having the subscription to the catalog, displaying the play button in the message.
 - 11.** The method of claim 9, further comprising: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; determining, by the messaging application, whether a user of the first device is signed-in to the catalog; responsive to the user of the first device having the subscription to the catalog and being signed-in to the catalog, displaying the play button in the message; and responsive to the user of the first device having the subscription to the catalog but being signed-out to the catalog, displaying the preview button in the message.
 - 12.** The method of claim 9, further comprising: determining, by the messaging application, whether a user of the first device has a subscription to the catalog; and responsive to the user of the first device not having the subscription to the catalog, displaying the preview button in the message.
 - 13.** The method of claim 12, further comprising displaying, in the message, a subscription option that is selectable to allow the user to start a subscription to the catalog.
 - 14.** The method of claim 9, wherein the play button or the preview button is displayed by a media extension application associated with the catalog executing on the first device.
 - 15.** The method of claim 9, wherein the play button or the preview button is displayed even if a media extension application associated with the catalog is not installed on the first device.
 - 16.** The method of claim 9, further comprising, based on the metadata, displaying artwork associated with the media content in the message.
 - 17.** An electronic device comprising: a display; and at least a memory and a processor to implement a messaging application, the messaging application configured to: receive a message from an additional electronic device, the message including metadata about media content in a catalog; display the message in a user interface; and display, based on the metadata, a play button or a preview button within the message, the play button selectable to cause playback of the media content and the preview button selectable to cause playback of a preview of the media content.
 - 18.** The electronic device of claim 17, wherein the messaging application is further configured to: determine whether a user of the electronic device has a subscription to the catalog; and responsive to the user of the electronic device having the subscription to the catalog, display the play button in the message.
 - 19.** The electronic device of claim 17, wherein the messaging application is further configured to: determine whether a user of the electronic device has a subscription to the catalog; determine whether a user of the electronic device is signed-in to the catalog; responsive to the user of the electronic device having the subscription to the catalog and being signed-in to the catalog, display the play button in the message; and responsive to the user of the electronic device having the subscription to the catalog but being signed-out to the catalog, displaying the preview button in the message.
 - 20.** The electronic device of claim 17, wherein the messaging application is further configured to: determine whether a user of the electronic device has a subscription to the catalog; and responsive to the user of the electronic device not having the subscription to the catalog, display the preview button in the message.
-